HCM CITY UNIVERSITY OF TECHNOLOGY

ME4204 - Introduction to Modern Control System

CURRICULUM REPORT

And Supervisory Reinforcement Learning Agent

Student Name	: Nguyễn Minh Hoàng
Student ID	: 2252234
Lecturing Professor	: Nguyễn Duy Anh
Practice Professor	: Trương Ngọc Cường

HCM CITY, May 2026

Table of Contents

I. Overview 1

1. The Inverted Pendulum System Model and Dynamics 1
2. The Coupled MSD System Model and Dynamics 1
3. Control Methodologies 2
4. Evaluation Model 4

II. Simulink models 5

1. The Inverted Pendulum Model 5
2. The Coupled MSD System Model 6

III. Simulations of Classical and Modern controls on the Inverted Pendulum system 7

1. Dual PID 7
2. SMC 10
3. Overall results 12

IV. Simulations of Classical and Modern controls on the CMSD system 13

1. PID with Feedforward 13
2. SMC 15
3. Adaptive SMC 17
4. Model Predictive Control 19
5. Other controllers 22

V. The case for Reinforcement Learning 25

VI. Training and Validation Environment 27

1. “Reward” and “Kill switch” 27
2. The Neural Network Training Script 32
3. Simulink training environment 38
4. Training and benchmark against other agents 40
5. Benchmarking and Stability against fixed-parameters SMC 47

VII. The Pros and Cons of RL Agent Training 59

I. Overview

1. The Inverted Pendulum System Model and Dynamics

The plant under control is an Inverted Pendulum on a Cart system, a 4-state mechatronic architecture. The system consists of a cart (mass *M*) restricted to horizontal 1D motion, and a pendulum (mass *m*, length to center of mass *l*, moment of inertia *I*) attached via a free pivot. The control effort *u* is applied directly to the cart *M*, making this a non-collocated control problem where the primary variable of interest (the pendulum angle θ) is physically separated from the actuator.

The Transfer Function:

To analyze the system in the frequency domain, we derive the transfer function $G(s) = \frac{\theta(s)}{U(s)}$

Assuming small angle approximations ($\sin(\theta) \approx 0; \cos(\theta) \approx 1; \theta^2 \approx 0$) and applying Laplace transforms to the linearized equations of motion yields:

$$(M+m)s^2X(s) + mls^2\theta(s) = U(s)$$

$$mls^2X(s) + (I+ml^2)s^2\theta(s) - mgl\theta(s) = 0$$

Solving for $\theta(s)$ relative to $U(s)$gives the general transfer function:

$$G(s) = \frac{-ml}{[(M+m)(I+ml^2) - (ml)^2]s^2 - (M+m)mgl}$$

The denominator reveals a sign difference between the s^2 coefficient and the constant term mgl. This indicates a pole in the right-half plane, pointing out the inherently unstable dynamics that the controller must manage to achieve stable balancing.

2. The Coupled MSD System Model and Dynamics

The plant under control is a Coupled Mass-Spring-Damper (CMSD) system, a 4-state mechatronic architecture representative of active suspension systems or dual-mass industrial positioning stages. The system consists of two masses (m_1 and m_2), connected to each other and to a fixed reference frame by springs (k_1, k_2) and dampers (b_1, b_2) The control effort $u(t)$ is applied directly to m_1, making this a non-collocated control problem where the primary variable of interest (the position of m_2) is physically separated from the actuator.

The Transfer Function:

To analyze the system in the frequency domain, we derive the transfer function $G(s) = \frac{X_2(s)}{U(s)}$

Applying Laplace transforms to the differential equations of motion yields:

$$m_1s^2X_1(s) + (b_1 + b_2)s\,X_1(s) + (k_1 + k_2)X_1(s) - b_2s\,X_2(s) - k_2X_2(s) = U(s)$$

$$m_2s^2X_2(s) + b_2s\,X_2(s) + k_2X_2(s) - b_2s\,X_1(s) - k_2X_1(s) = 0$$

Solving for $X_2(s)$relative to $U(s)$ gives the general transfer function:

$$G(s) = \frac{(b_2 s + k_2)}{[(m_1 s^2 + (b_1 + b_2)s + (k_1 + k_2)][m_2 s^2 + b_2 s + k_2) - (b_2 s + k_2)^2]}$$

The 4th-order denominator is the complex, resonant internal dynamics that the controller must manage to achieve stable tracking.

3. Control Methodologies

To evaluate optimal performance, seven distinct control architectures were implemented and tested:

Proportional-Integral-Derivative (PID) with Feedforward:

- *Definition:* A model-free controller that calculates effort based on the present (P), past (I), and predicted future (D) errors.
- *Pros:* Simple to tune, universally understood, highly precise for static loads.
- *Cons:* Reactive rather than predictive; entirely blind to the internal physics (like the spring resonances) of the CMSD system.
- *Note: Feedforward Mechanism:* A feedforward term calculates and injects the exact baseline force required to hold a position (e.g., fighting gravity or steady-state spring tension), leaving the PID feedback loop responsible only for correcting transient errors.

Adaptive PID:

- *Definition:* A PID controller where the , and gains dynamically shift based on operating conditions or error thresholds.
- *Pros:* Can adjust to varying payloads (e.g., changes to).
- *Cons:* High computational overhead; can become unstable if the adaptation mechanism overreacts.

Linear Quadratic Regulator (LQR):

- *Definition:* An optimal, model-based state-feedback controller. It uses a cost function () to find the perfect mathematical balance between state errors (Q matrix) and energy usage (R matrix).
- *Pros:* Highly efficient; mathematically guarantees stability by observing all 4 states simultaneously.
- *Cons:* Without an integral term, it acts purely as a regulator (driving to zero) and struggles with non-zero tracking.

PID + LQR Hybrid:

- *Definition:* A cascaded architecture where LQR handles the internal stabilization (inner loop) and PID forces the setpoint tracking (outer loop).
- *Pros:* Best of both worlds conceptually.
- *Cons:* Complex to tune; the two loops can "fight" each other, leading to high energy spikes.

Sliding Mode Control (SMC):

- *Definition:* A robust, non-linear controller that defines a "sliding surface" in the state space and uses high-speed switching to force the system onto that surface, rendering it immune to matched disturbances.
- *Pros:* Extreme robustness; theoretically perfect tracking even with unknown payloads or heavy disturbances.
- *Cons:* "Chattering" (high-frequency vibrations) which can damage mechanical actuators if not smoothed by a boundary layer.

Adaptive SMC:

- *Definition:* An SMC where the switching gain (η) or the boundary layer (ϕ) adapts in real-time. In this case, our algorithm is based on the Culture-Bees Algorithm, where α, λ, μ (corresponding to η, C, ϕ) are re-optimized at every 0.01s sample by running the full Culture-Bees search.
- This algorithm mimics the foraging behavior of a honeybee colony. In this metaphor, the "food sources" are different combinations of controller gains (α, λ, μ). A population of "scout bees" explores the mathematical landscape, while "onlooker bees" flock toward the combinations that yield the lowest tracking error, effectively "foraging" for the most efficient control law.
- Unlike standard optimization, CBA includes a "Belief Space". This acts as a collective memory of the colony, hence the "Culture" in "Culture-Bee". By storing the "best" gains found in previous samples, the algorithm doesn't have to start its search from scratch every 0.01 seconds; it uses its "culture" to refine its search near previously successful areas, ensuring the adaptation is fast enough for real-time mechatronic control.
- *Pros:* Maximizes precision when far from the target while aggressively smoothing the signal near the target to kill chattering.
- *Cons:* Running a full search algorithm every 10 milliseconds requires significant processing power compared to fixed-gain laws. Furthermore, the "meta-parameters" of the bee colony itself (population size, search limits) require their own expert tuning to ensure the search converges before the next sample arrives.

Model Predictive Control (MPC):

- *Definition:* A control strategy that uses a mathematical model of the system to predict future behavior over a specific time horizon. At each time step, it solves an online optimization problem to find the best sequence of control inputs that minimizes a cost function while strictly respecting system constraints (e.g., motor torque limits or maximum velocity).
- *Pros:* Inherently handles multi-variable systems and complex constraints; provides "look-ahead" capability to preemptively adjust for upcoming setpoint changes or known disturbances, resulting in highly efficient and smooth trajectories.
- *Cons:* Computationally expensive, as it requires solving an optimization algorithm in real-time at every sampling interval. Furthermore, performance is heavily dependent on the accuracy of the internal system model.

4. Evaluation Model

Performance is judged on two primary vectors: Tracking Precision (x_2) and Actuator Wear/Energy (u). The main metrics used are:

Mean Absolute Error (MAE): The average absolute deviation from the target. Used to define the overall accuracy of x_2 and the average continuous power draw of u.

Formula:

$$MAE = \frac{1}{n}\sum |Actual - Target|$$

Root Mean Squared Error (RMSE): Squares the error before averaging, heavily penalizing large deviations. Applied to x_2, it highlights unwanted oscillations/bouncing. Applied to u, it highlights violent, high-frequency motor spikes that cause mechanical fatigue.

Formula:

$$MSE^2 = \frac{1}{n}\sum (Actual - Target)^2$$

$$RMSE = \sqrt{MSE}$$

II. Simulink models

1. The Inverted Pendulum Model

The Inverted Pendulum subsystem was constructed in Simulink to serve as a benchmark between a classical control scheme and a modern control scheme. The Inverted Pendulum was picked due to its inherent and often violent instability. The cart's travel limit was set at $\pm 1m$.

Due to the instability of the system, a Rate Limiter block (± 1 slew rate) was placed between the Reference change command blocks and the SMC. No limit was placed on acceleration of the cart as this is done purely for benchmarking reasons.

Cart Mass: $M = 2.4\ kg$

Pendulum Mass: $m = 0.23\ kg$

Distance to Pendulum Center of Mass: $l = 0.36\ m$

Gravity: $g = 9.8\ m/s^2$

The control input (u) acts directly on M, the unit of this control signal u is in *meters*.

All systems are fitted with a "Band-limited White noise" block with noise power 0.001 (m), 0.01s sampling time and seed: 23341 to simulate sensor noise.

The functions:

$$x'' = \frac{u_1 + 0.23 + 0.36\sin(u_2)\,u_3^2 - 0.23\ \cdot 9.8\cos(u_2)\sin(u_2)}{2.4 + 0.23 - 0.23\cos^2(u_2)}$$

$$\theta'' = \frac{u_1\cos(u_2) - (2.4 + 0.23)\cdot 9.8\sin(u_2) + 0.23\cdot 0.36\cos(u_2)\sin(u_2)\,u_3^2}{0.23\cdot 0.36\cos^2(u_2) - (2.4 + 0.23)\cdot 0.36}$$

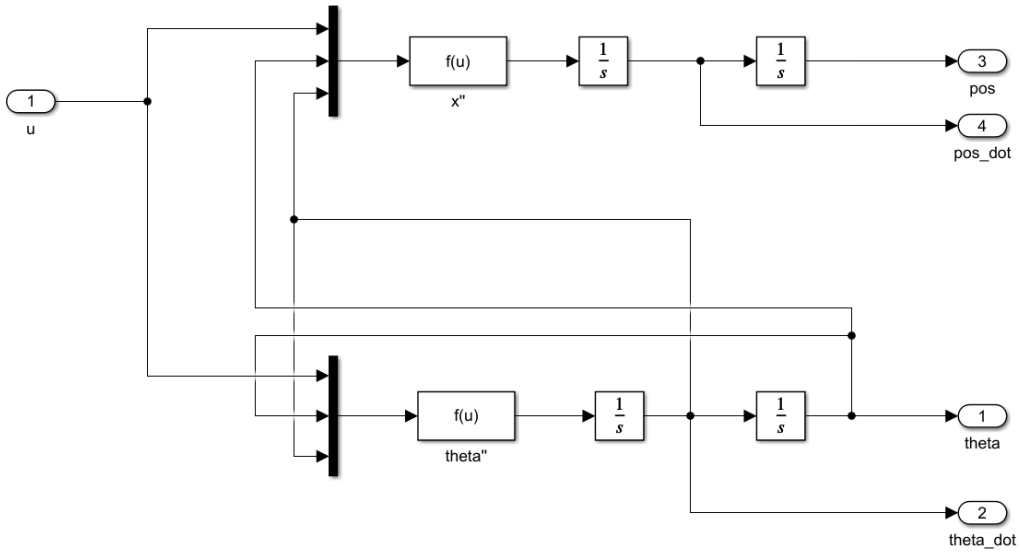


Figure 2.1: Simulink model of the Inverted Pendulum system

2. The Coupled MSD System Model

The CMSD subsystem was constructed in Simulink to represent the active suspension of a heavy vehicle.

Unsprung Mass (Wheel/Axle):, $k_1 = 750\ N/m$, $b_1 = 50\ Ns/m$

Sprung Mass (Chassis/Load): $m_2 = 300\ kg$, $k_2 = 250\ N/m$, $b_2 = 100\ Ns/m$

The control input (u) acts directly on m_1, simulating an actuator placed between the ground and the chassis. This creates a "non-collocated control" problem, as the primary variable of interest (x_2) is separated from the actuator by a flexible spring (k_2).

All systems are fitted with a "Band-limited White noise" block with noise power 200 (N), 0.01s sampling time and seed: 23341. And the Control input power is capped at [-3500; 3500] (N).

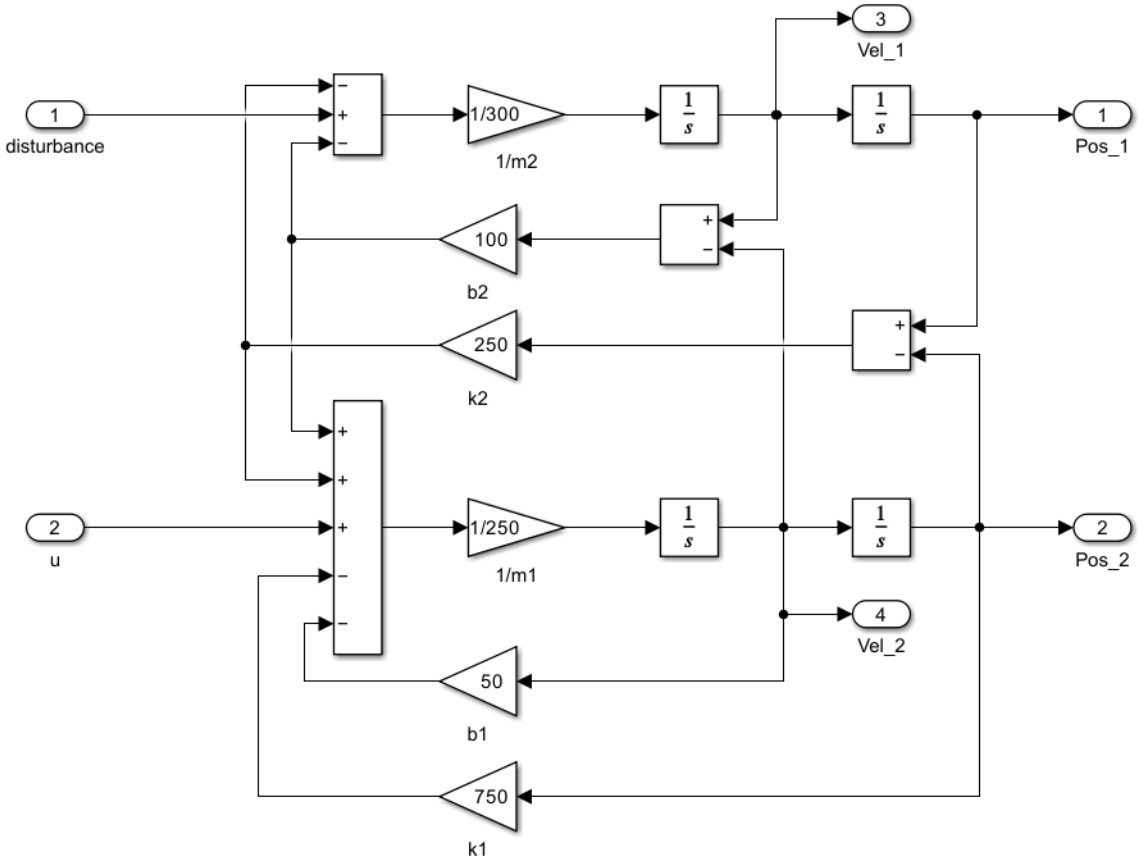


Figure 2.2: Simulink model of the Coupled Mass Spring Damper system

III. Simulations of Classical and Modern controls on the Inverted Pendulum system

1. Dual PID

a. Model

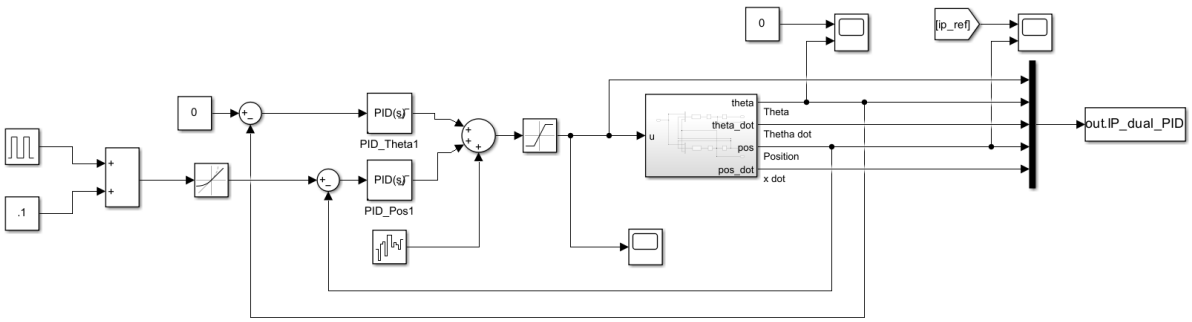


Figure 3.1: Dual-PID Control model for the IP system

The Controller's parameters are as follows:

For PID_Theta: $K_P = -40; K_I = 0; K_D = -8$.

For PID_Pos: $K_P = -1.25; K_I = 0; K_D = -3.6$.

An inverted pendulum is a **Underactuated System**. We have two degrees of freedom (cart position x and pendulum angle θ), but only one actuator: the motor pushing the cart.

Because moving the cart causes the pendulum to fall, these two controllers are actively fighting each other. The sum block at the end is just a brute-force compromise. It works only when the pendulum is already nearly perfectly vertical (the linear region). If the angle gets too wide, the system will crash.

b. Result Diagrams

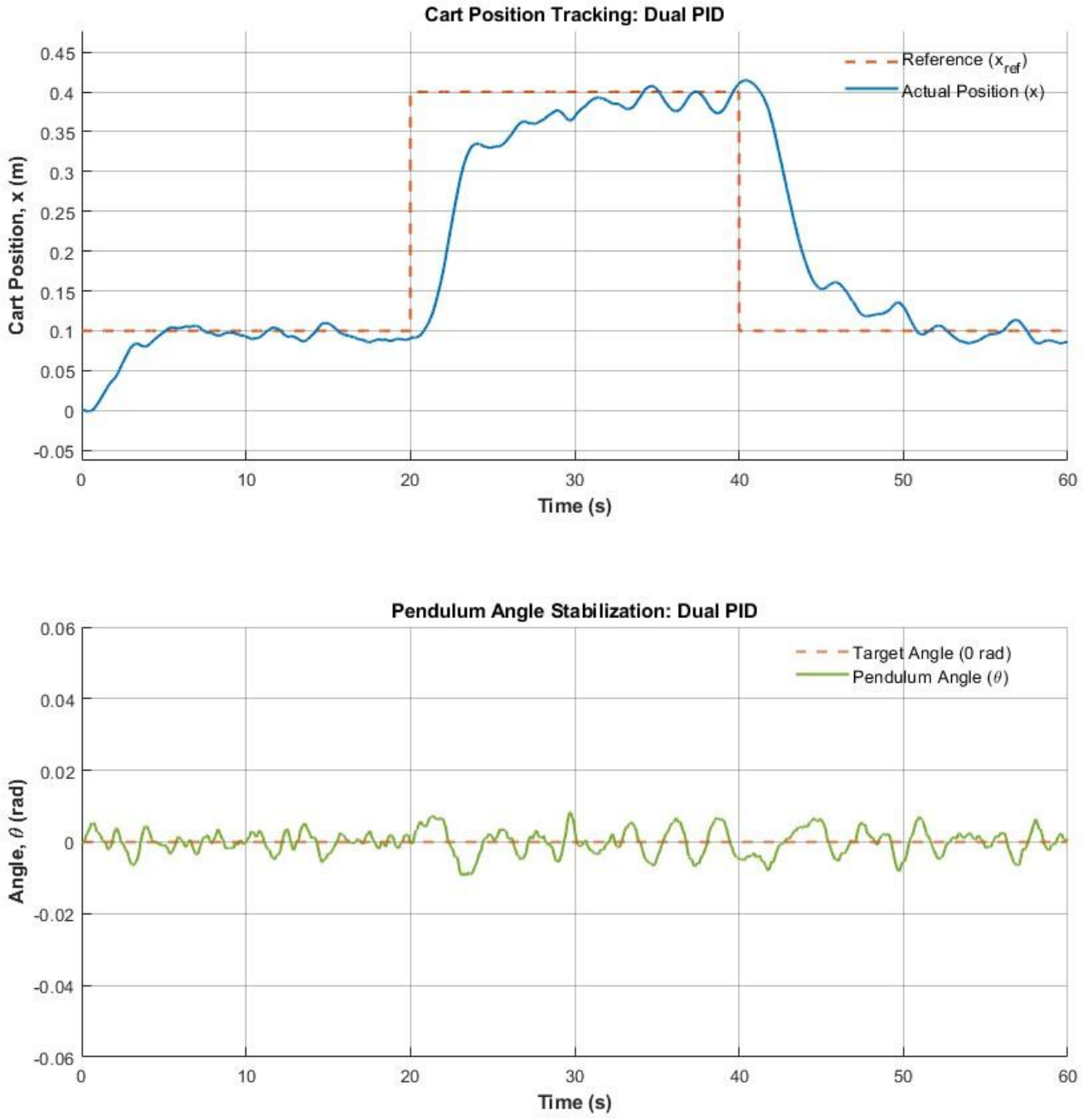


Figure 3.2: Dual-PID Control overall tracking performance

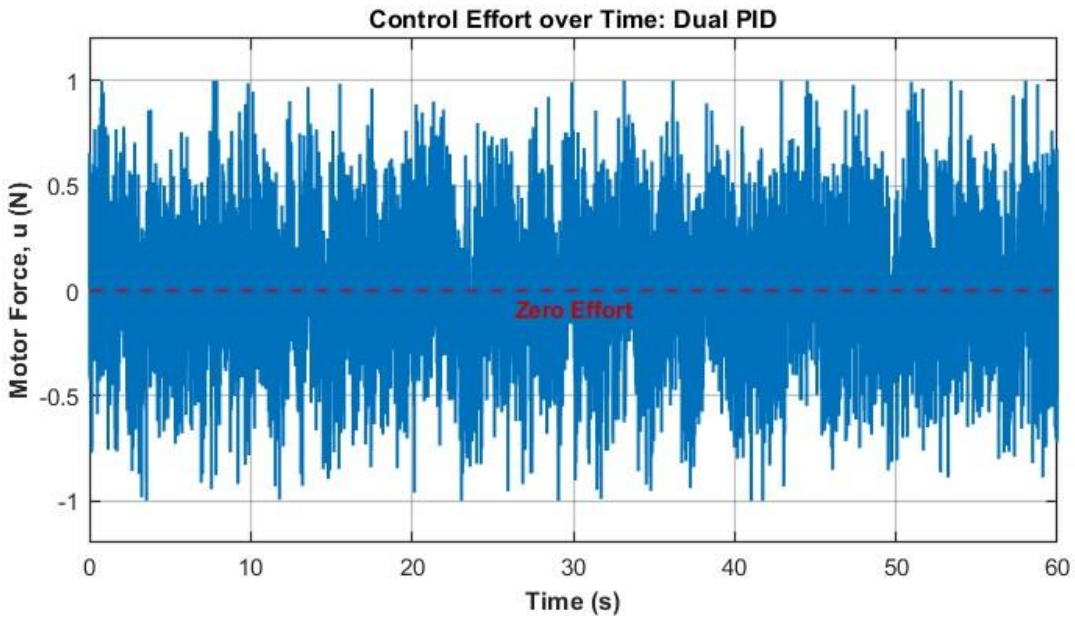


Figure 3.3: Dual-PID Control cart position over time

c. Numerical Results & Verdict

Tracking (x): MAE = 0.0582 m | RMSE = 0.1068 m

Tracking (θ): MAE = 0.0028 rad | RMSE = 0.0035 rad

Effort (u): MAE = 0.2874 m | RMSE = 0.3628 m

Verdict: The Dual PID manages to successfully keep the system afloat but suffers from the architectural flaw of decoupling inherently coupled dynamics. Because the position loop and angle loop actively fight each other during transient moves, it results in noticeably looser position tracking and angle regulation. It achieves this baseline stability while utilizing the marginally lowest control effort of the two setups.

2. SMC

a. Model

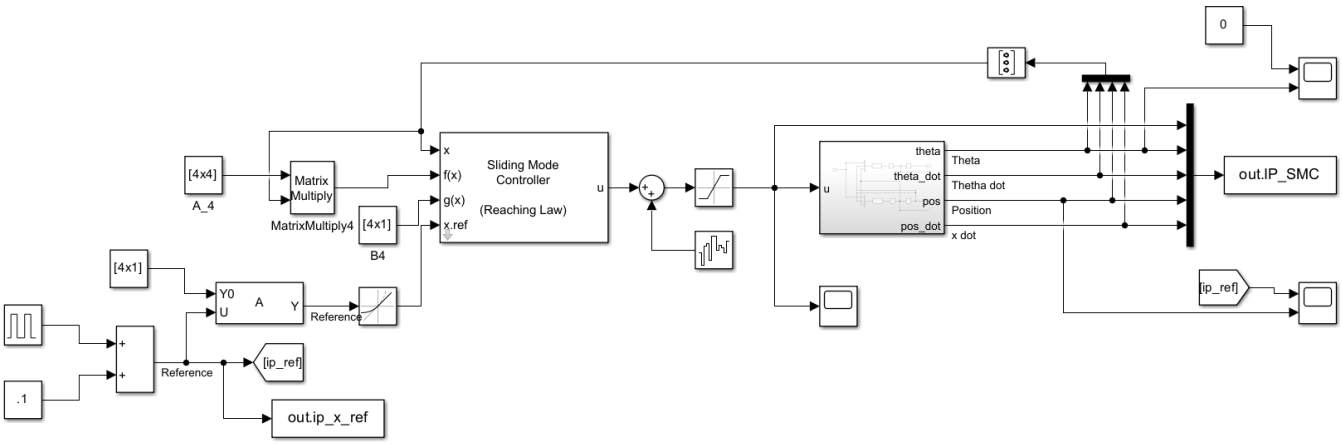


Figure 3.4: SMC model for the IP system

The Controller's parameters are as follows:

For the Sliding Matrix: $C = [-654.4437;\ -148.1602;\ -100;\ -117.5960]$.

For Reaching Law (Exponential): $\eta = 100; K = 10$.

For Boundary layer (Hyperbolic tangent): $\phi = 1$.

The sliding matrix is obtained by the code below:

```
% State vector order: [theta; theta_dot; x; x_dot]
A = [0, 1, 0, 0;
     29.8310, 0, 0, 0;
     0, 0, 0, 1;
     -0.9392, 0, 0, 0];

B = [0; -1.1574; 0; 0.4167];

% Penalties
% Massive penalty on angle (1000), moderate on position (100)
Q = diag([1000, 100, 100, 10]);
R = 0.01;

% 3. Calculate Optimal LQR Gain
K = lqr(A, B, Q, R);

% 4. Acquire the Sliding Matrix C
C = K;

% Display
disp('Sliding Matrix C is:');
disp(C);
```

b. Result Diagrams

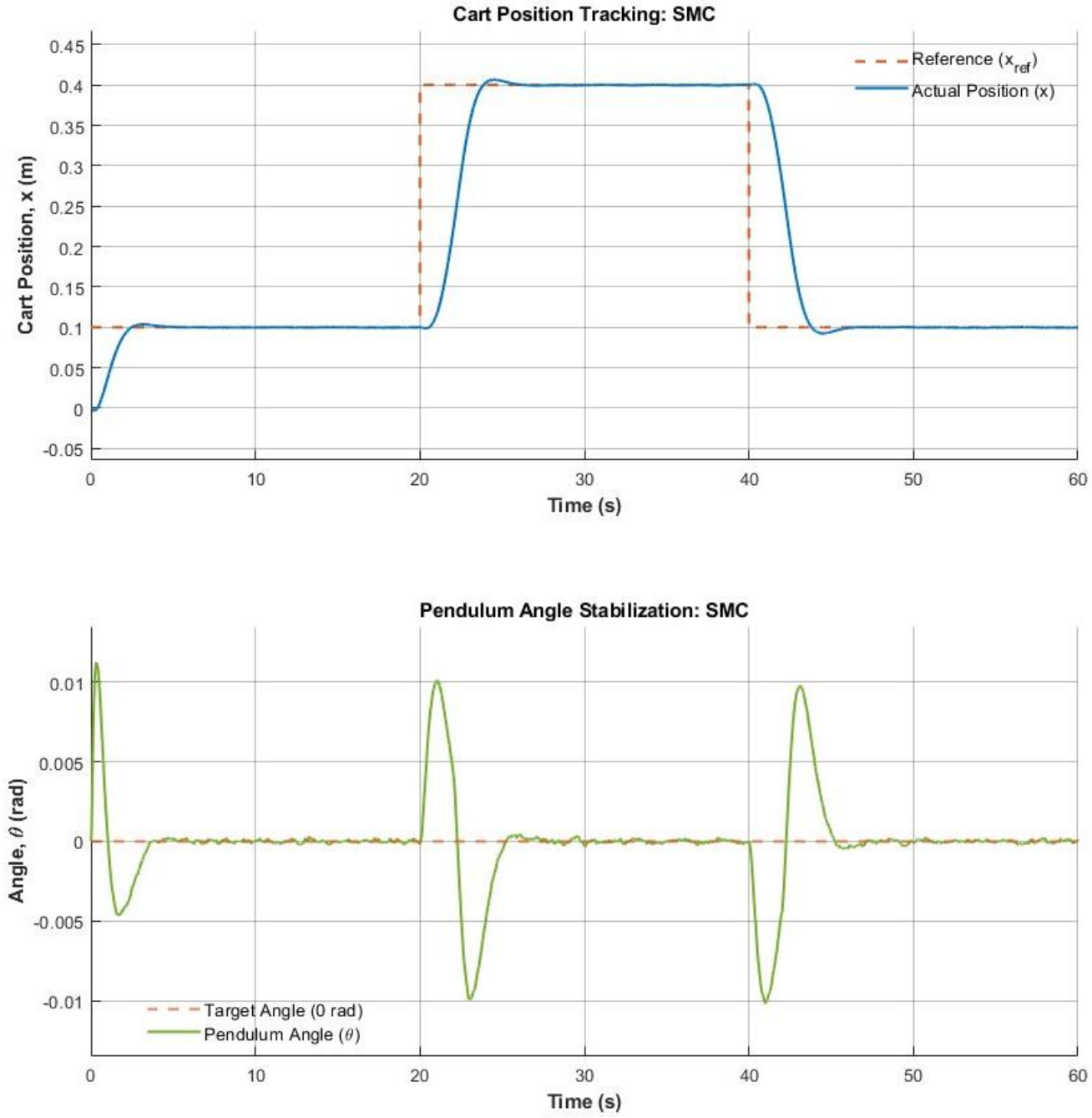


Figure 3.5: SMC overall tracking performance

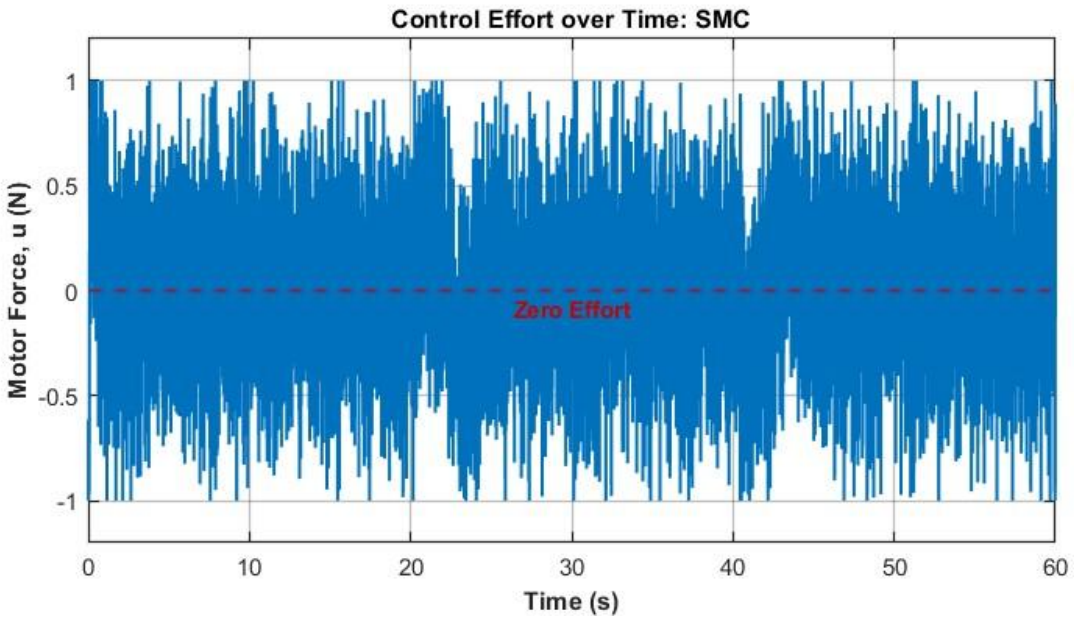


Figure 3.6: SMC cart position over time

a. Numerical Results & Verdict

Tracking (x): MAE = 0.0342 m | RMSE = 0.0902 m

Angle (θ): MAE = 0.0015 rad | RMSE = 0.0032 rad

Effort (u): MAE = 0.2935 m | RMSE = 0.3829 m

Verdict: The SMC is a performance upgrade over the classical approach. By utilizing a single sliding surface, it decreases position tracking error (MAE) by 41.2% and improves the pendulum's balance precision (Angle MAE) by 46.7% compared to the Dual PID. Most importantly, it achieves this very notable gain in stability and tracking with only a negligible 2.1% increase in average control effort (MAE) and a 5.5% higher effort variance (RMSE), making it vastly superior for non-linear trajectory tracking.

3. Overall results

	MAE	MSE	RMSE
Dual PID	0.002818595	1.24213E-05	0.003524384
SMC	0.001502782	1.04879E-05	0.003238497

Table 1: The Controllers' angle tracking performance

	MAE	MSE	RMSE
Dual PID	0.0582153	0.011398062	0.106761708
SMC	0.034223701	0.008134921	0.090193795

Table 2: The Controllers' position tracking performance

Row	MAE	MSE	RMSE
Dual PID	0.287376502	0.131647393	0.362832459
SMC	0.293542512	0.146614626	0.382902894

Table 3: The Controllers' average cart position

IV. Simulations of Classical and Modern controls on the CMSD system

1. PID with Feedforward

a. Model

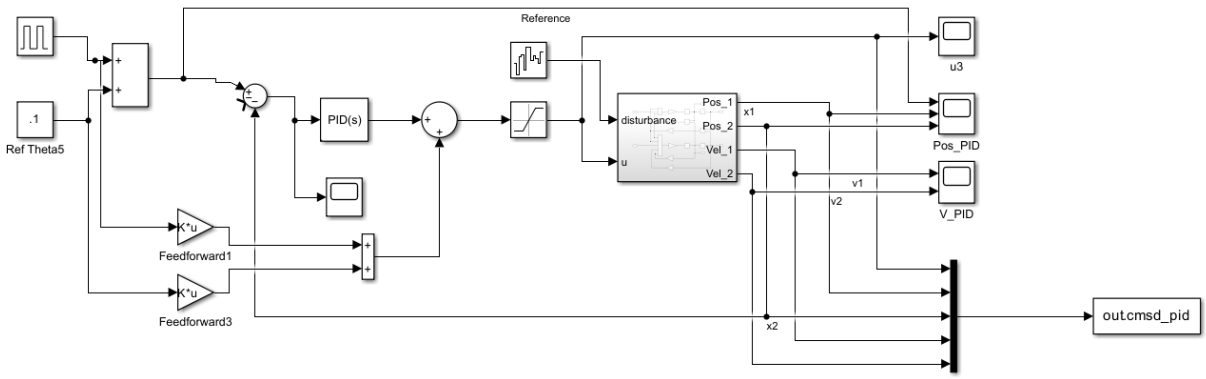


Figure 4.1: PID Control model for the CMSD system

The Controller's parameters are as follow: $K_P = 12000; K_I = 1100; K_D = 5000$. With Feedforward Gains for the step command and the original 0.1 reference points being 200 and 300 respectively.

b. Result Diagrams

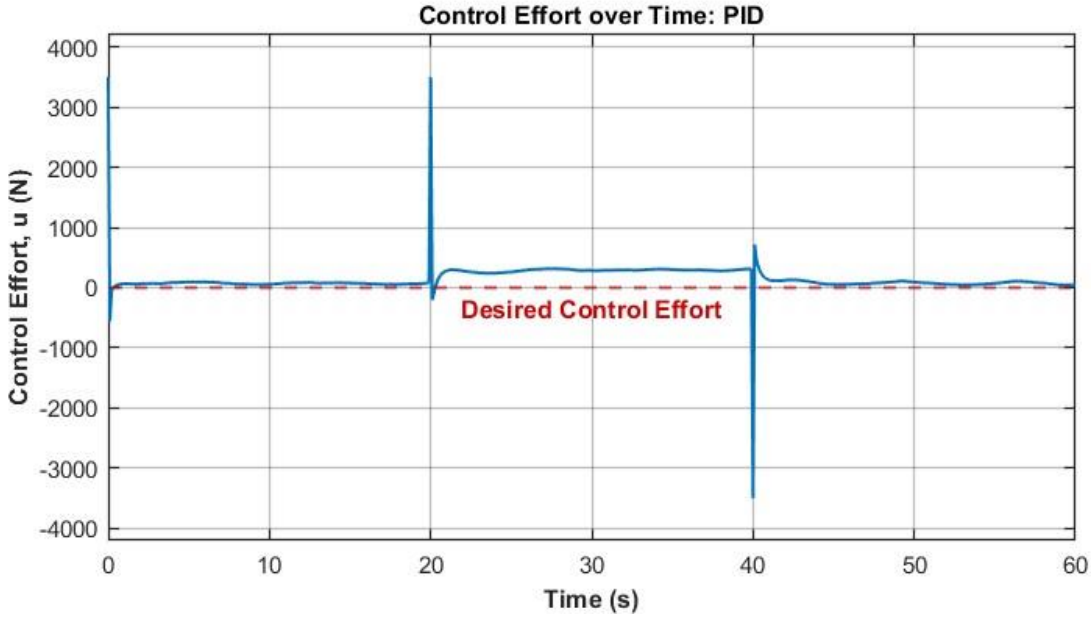


Figure 4.2: PID Control's Effort over time

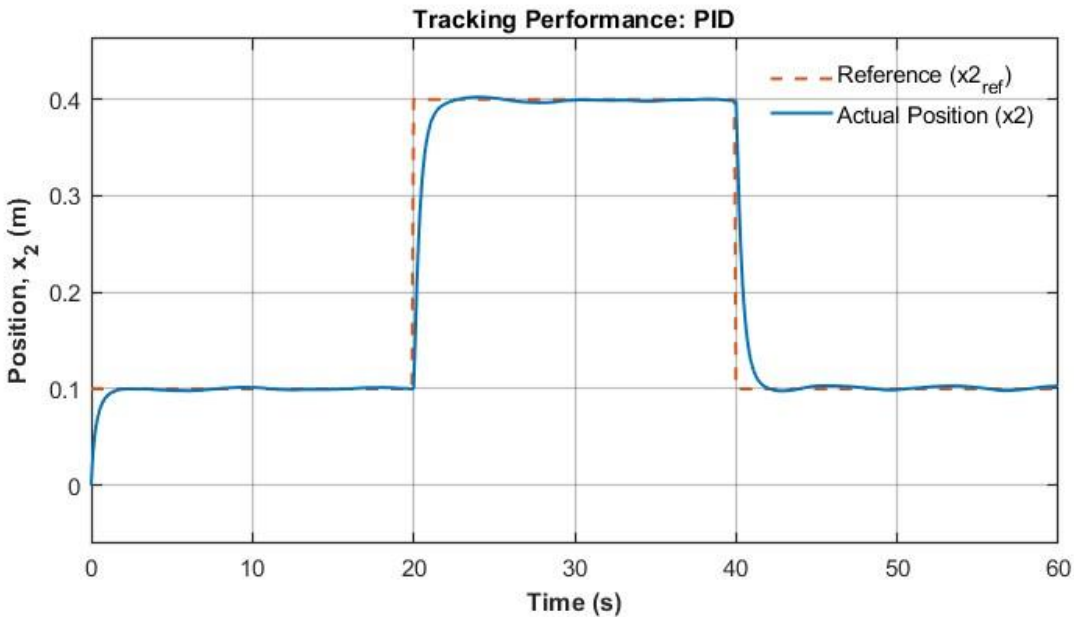


Figure 4.3: PID Tracking performance over time

c. Numerical Results & Verdict

Tracking (x_2): MAE = 0.0064 m | RMSE = 0.0288 m
Effort (u): MAE = 165.04 N | RMSE = 307.16 N

Verdict: Standard PID achieves very performant tracking precision and rise time but is rather prone to disturbance as seen in the graph. However, due to it lacking knowledge of the system's physics, it forces the error to zero by violently acting on the actuator, resulting in the highest mechanical stress of all tested controllers.

2. SMC

a. Model

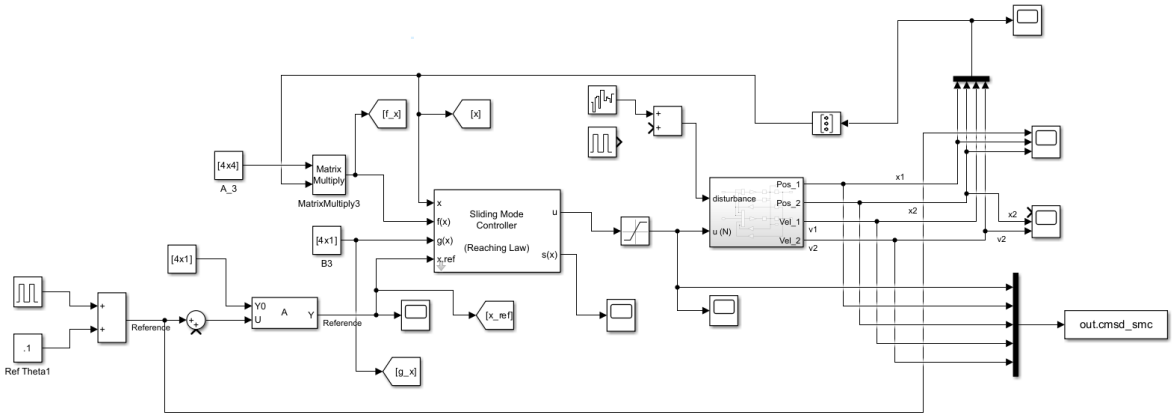


Figure 4.4: SMC model for the CMSD system

The Controller's parameters are as follows:

Sliding Coefficient Matrix: [0; 8; 0; 4]

Exponential Reaching Law: $\eta = 20; K = 10$

Hyperbolic Tangent boundary layer: $\phi = 0.0274$

b. Result Diagrams

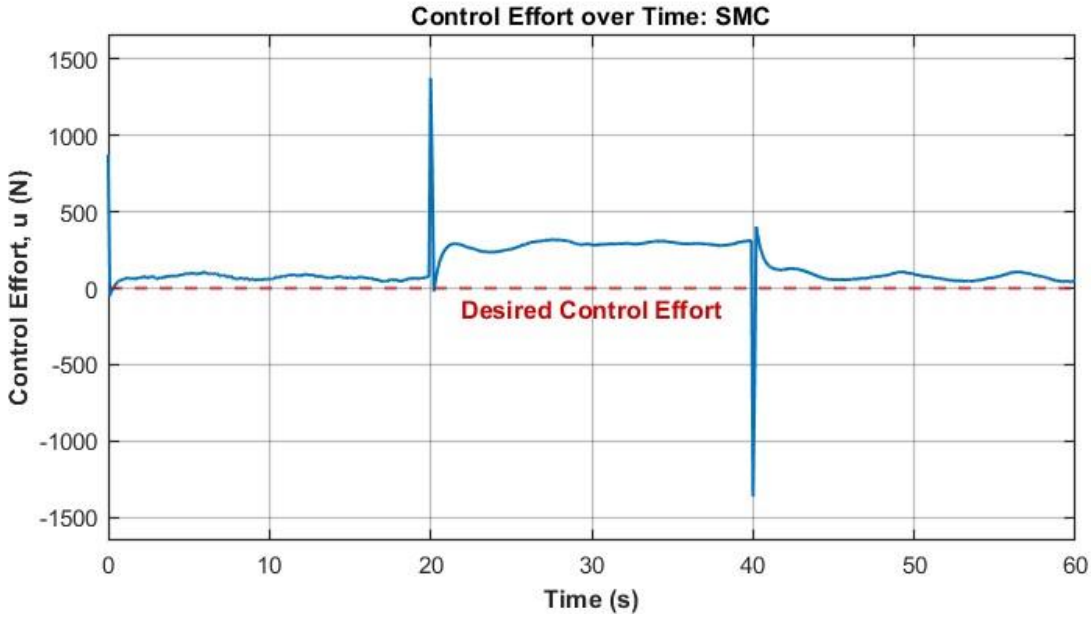


Figure 4.5: SMC's Effort over time

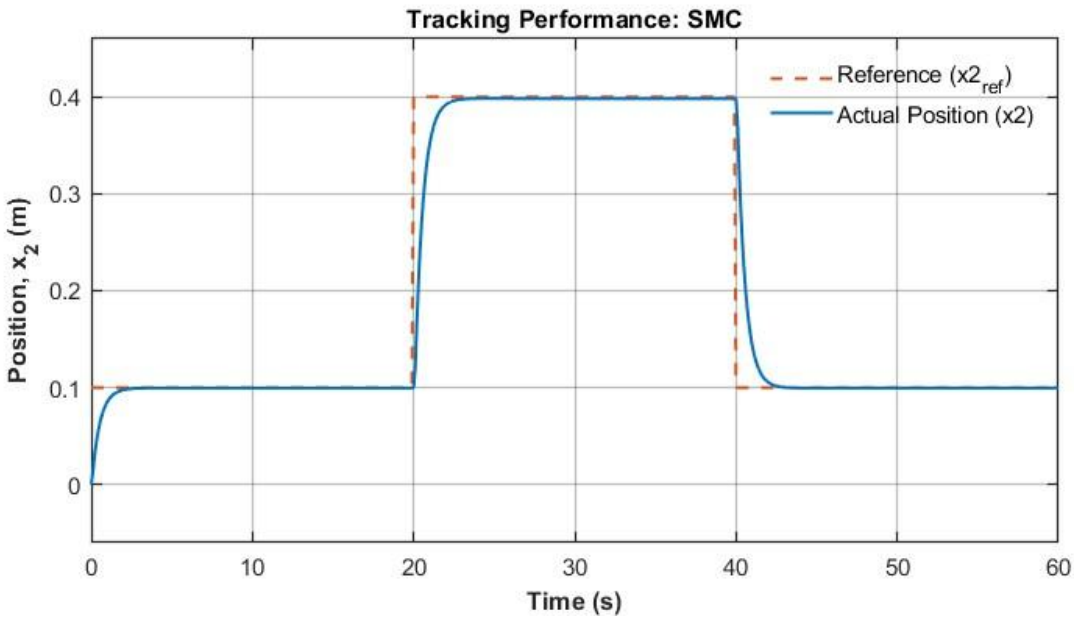


Figure 4.6: SMC Tracking performance over time

c. Numerical Results & Verdict

Tracking (x_2): MAE = 0.0077 m | RMSE = 0.0336 m
Effort (u): MAE = 153.00 N | RMSE = 201.33 N

Verdict: SMC proves to be highly resilient to disturbances as theoretical models suggest. By utilizing the A and B state-space matrices, it provides tracking precision that is 95% as accurate as pure PID, but requires roughly 34% less peak mechanical force (RMSE 201 vs 307) to achieve it.

3. Adaptive SMC

a. Model

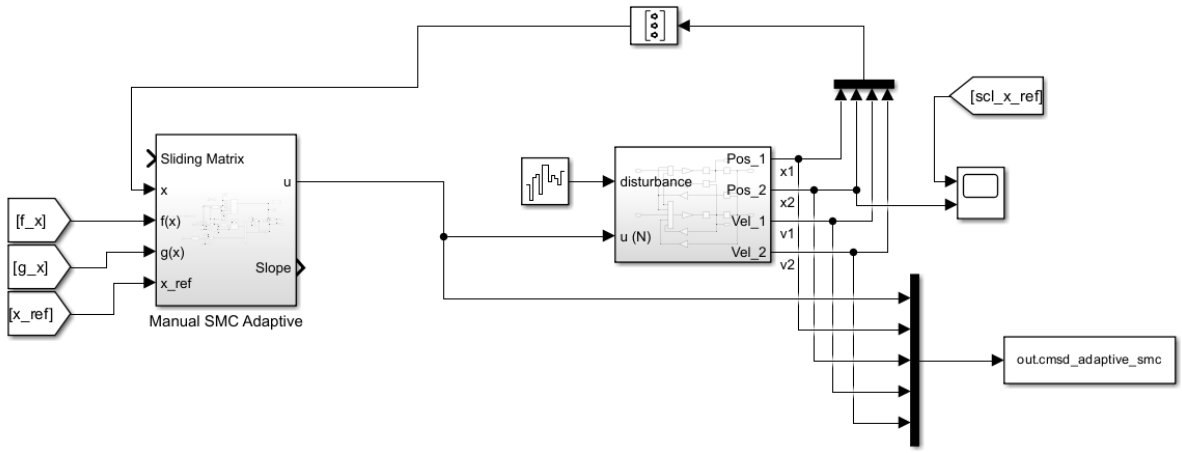


Figure 4.7: Adaptive SMC model for the CMSD system

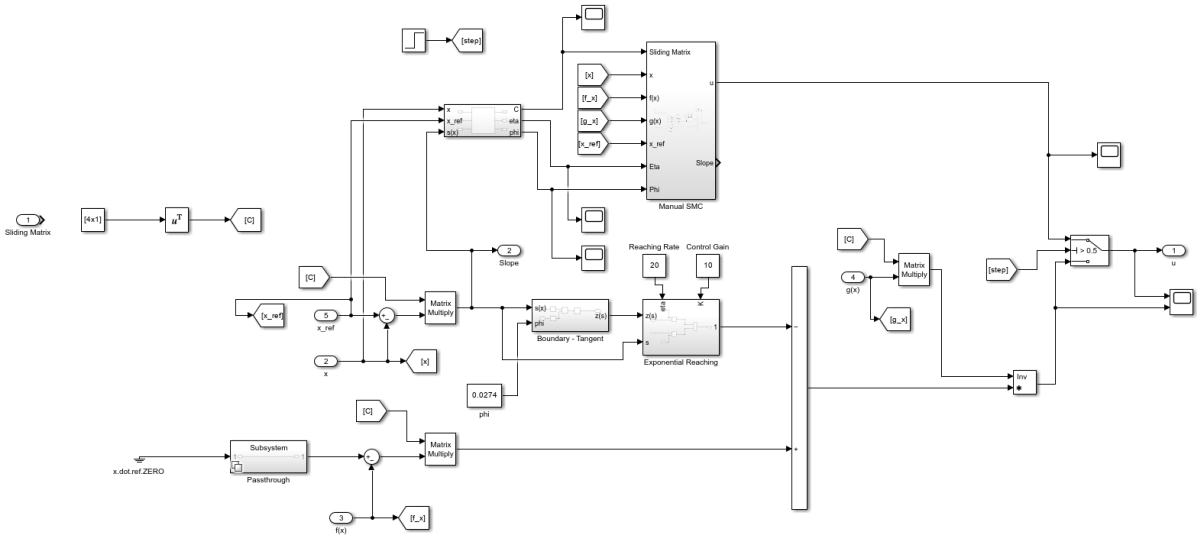


Figure 4.8: Adaptive SMC internal view

The Controller's parameters are as follows:

Sliding Coefficient Matrix: [0; 3÷15; 0; 3÷15]

Exponential Reaching Law: $\eta = 8 \div 25; K = 10$

Hyperbolic Tangent boundary layer: $\phi = 0.02 \div 0.1$

b. Result Diagrams

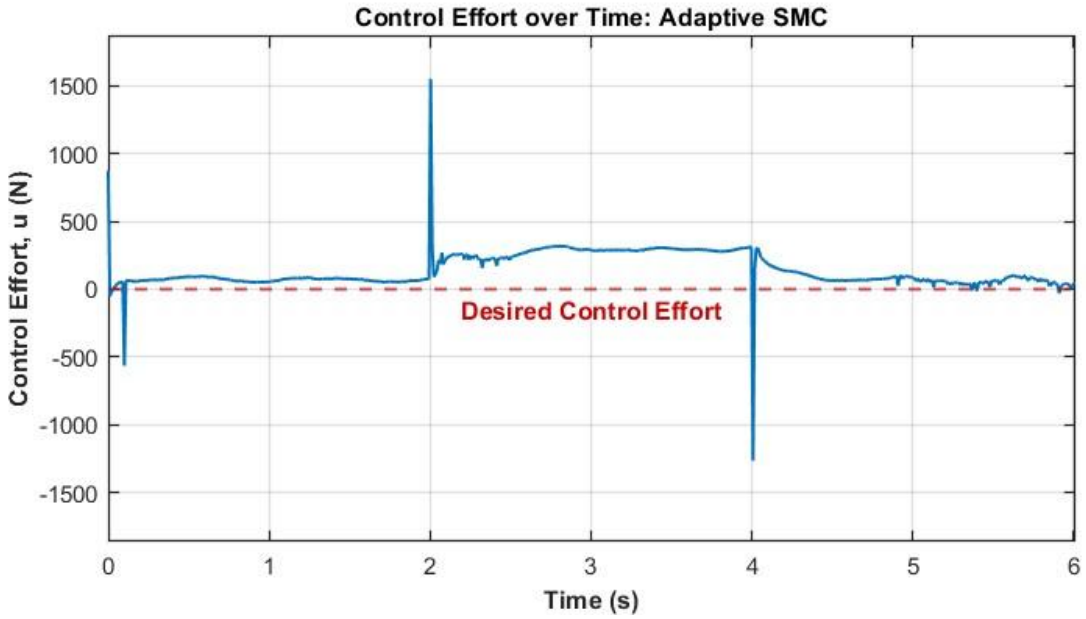


Figure 4.9: Adaptive SMC Control's Effort over time

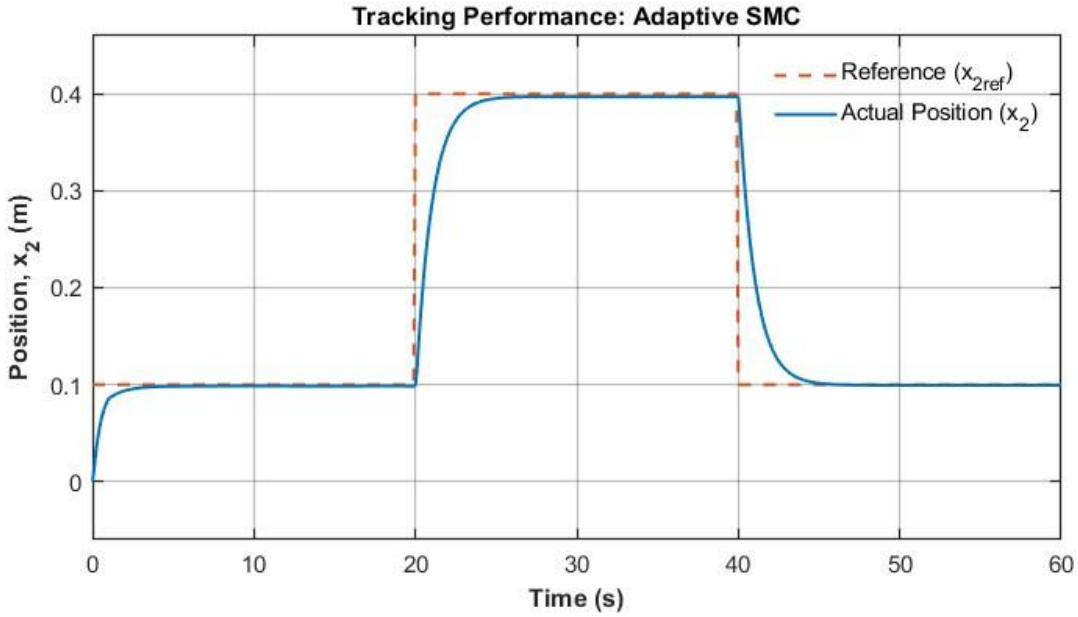


Figure 4.10: Adaptive SMC Tracking performance over time

c. Numerical Results & Verdict

Tracking (x_2): MAE = 0.0133 m | RMSE = 0.0425 m
Effort (u): MAE = 147.89 N | RMSE = 196.48 N

Verdict: Adaptive SMC achieves the second-lowest control effort (RMSE 196.48 N), confirming that the adaptive gain law successfully reduces unnecessary switching near the sliding surface. However, this gain reduction comes at a tracking cost: positional error is 26% higher than plain SMC (RMSE 0.043 vs 0.034 m). The adaptation parameters require further tuning to recover tracking precision without sacrificing the effort savings.

4. Model Predictive Control

a. Model

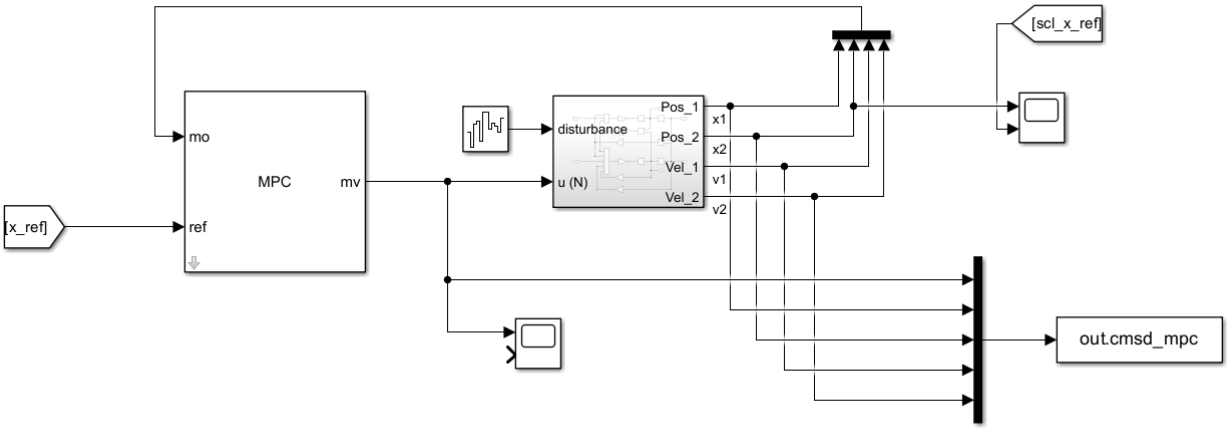


Figure 4.11: MPC model for the CMSD system

The Controller's parameters are as follows:

Sample time: 0.01; Prediction Horizon = 50; Control Horizon = 5

Constraints (for u): $u \in [-3500; 3500]; rate\ change\ \leq 500N/step$

Weight: $[x_1; x_2; v_1; v_2] = [1; 1e5; 5; 10]$

b. Result Diagrams

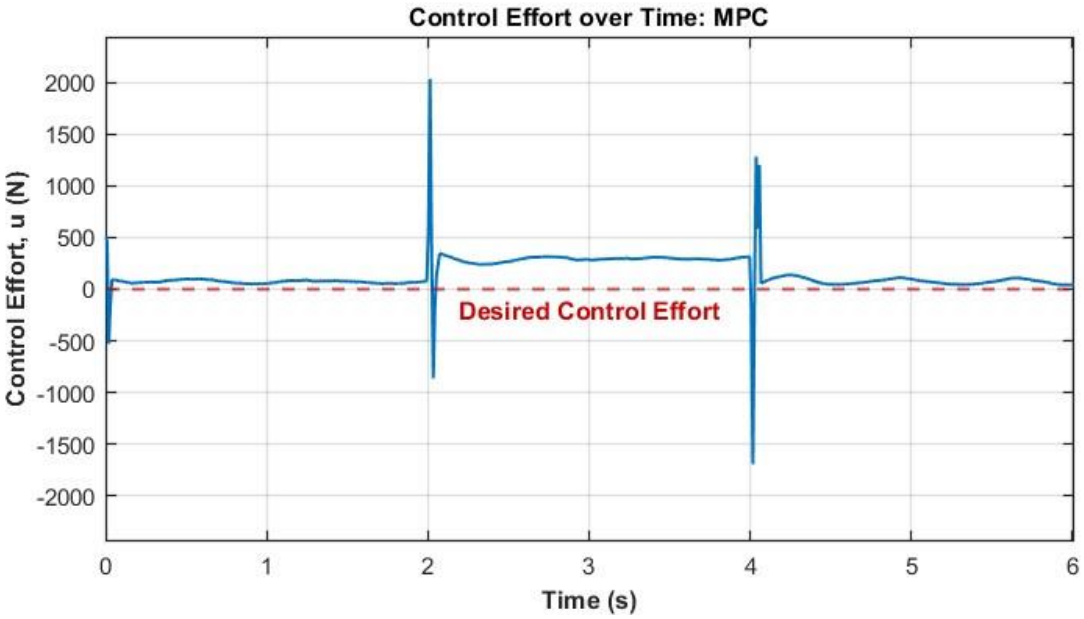


Figure 4.12: MPC's Effort over time

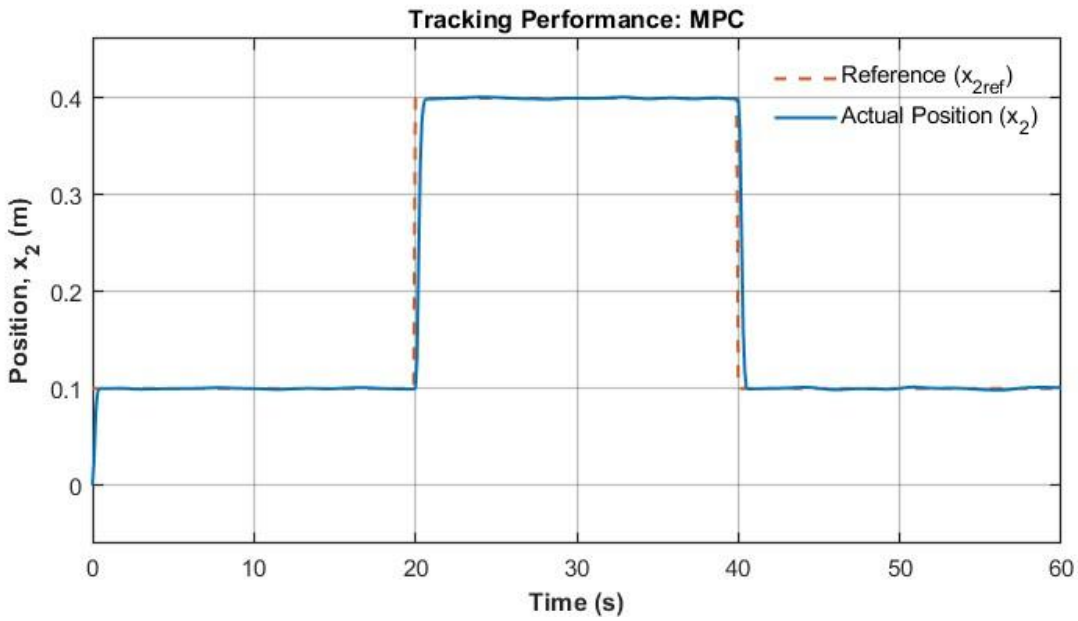


Figure 4.13: MPC Tracking performance over time

Tracking (x_2): MAE = 0.0036 m | RMSE = 0.0259 m
Effort (u): MAE = 160.23 N | RMSE = 228.01 N

Verdict: MPC delivers the best tracking performance of all controllers tested, with the lowest MAE (0.0036 m) and RMSE (0.0259 m) for position x_2. Its predictive horizon allows it to anticipate reference changes rather than react to them. This precision comes at a moderate effort cost (RMSE 228 N), 16% more than SMC but with 23% better tracking accuracy, a favorable trade-off for applications where positional precision is the primary objective.

5. Other controllers

For this particular test, only 4 controllers are shown as the remainer three (Adaptive PID, LQR, PID+LQR) were either: Barely more effective, even having worse performance due to insufficient tuning (Adaptive PID, PID+LQR, including the above Adaptive Culture-Bee SMC); losing out to the easier-to-tune SMC (PID+LQR); or simply too inaccurate on its own (LQR — due to the fact that it's a regulator, not a tracker controller in the common sense of the word).

However, their results will still be shown.

Note:

For the PID + LQR Hybrid controller, the LQR's matrices are as follows:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -4 & -0.6 & 1 & 0.4 \\ 0 & 0 & 5000 & 1 \\ 0.8333 & 0.3333 & -0.8333 & -0.3333 \end{bmatrix}; B = \begin{bmatrix} 0 \\ 0.004 \\ 0 \\ 0 \end{bmatrix}; Q = \begin{bmatrix} 100 & 0 & 0 & 0 \\ 0 & 400 & 0 & 0 \\ 0 & 0 & 5000 & 0 \\ 0 & 0 & 0 & 1000 \end{bmatrix}$$

$$R = [0.001];\ K = [197.1274;\ 543.9573;\ 131.0655;\ 963.0780]$$

For the LQR-only controller, the LQR's K matrix is as follows:

$$K = [242.4879\ \ 684.9939\ \ 85.7051\ \ 920.8251$$

The end data for the seven controllers are:

Controller	MAE	MSE	RMSE
PID	0.00643393	0.000834002	0.028879098
Adaptive PID	0.008990637	0.00093273	0.030540628
LQR	0.023367971	0.003225164	0.05679053
PID + LQR	0.010004985	0.001385962	0.037228511
SMC	0.0077977	0.001131123	0.033632179
Adaptive SMC	0.013268374	0.001806838	0.042506917
MPC	0.003576619	0.000670043	0.02588518

Table 4: The Controllers' tracking performance

Controller	MAE	MSE	RMSE
PID	165.0427821	94348.59303	307.1621608
Adaptive PID	163.7844578	94167.20196	306.8667495
LQR	144.56699	30763.42861	175.3950644
PID + LQR	165.4841899	94875.68446	308.0189677
SMC	151.5858922	40317.74979	200.7928031
Adaptive SMC	147.892297	38602.93293	196.476291
MPC	160.2307396	51990.36444	228.0139567

Table 5: The Controllers' energy usage

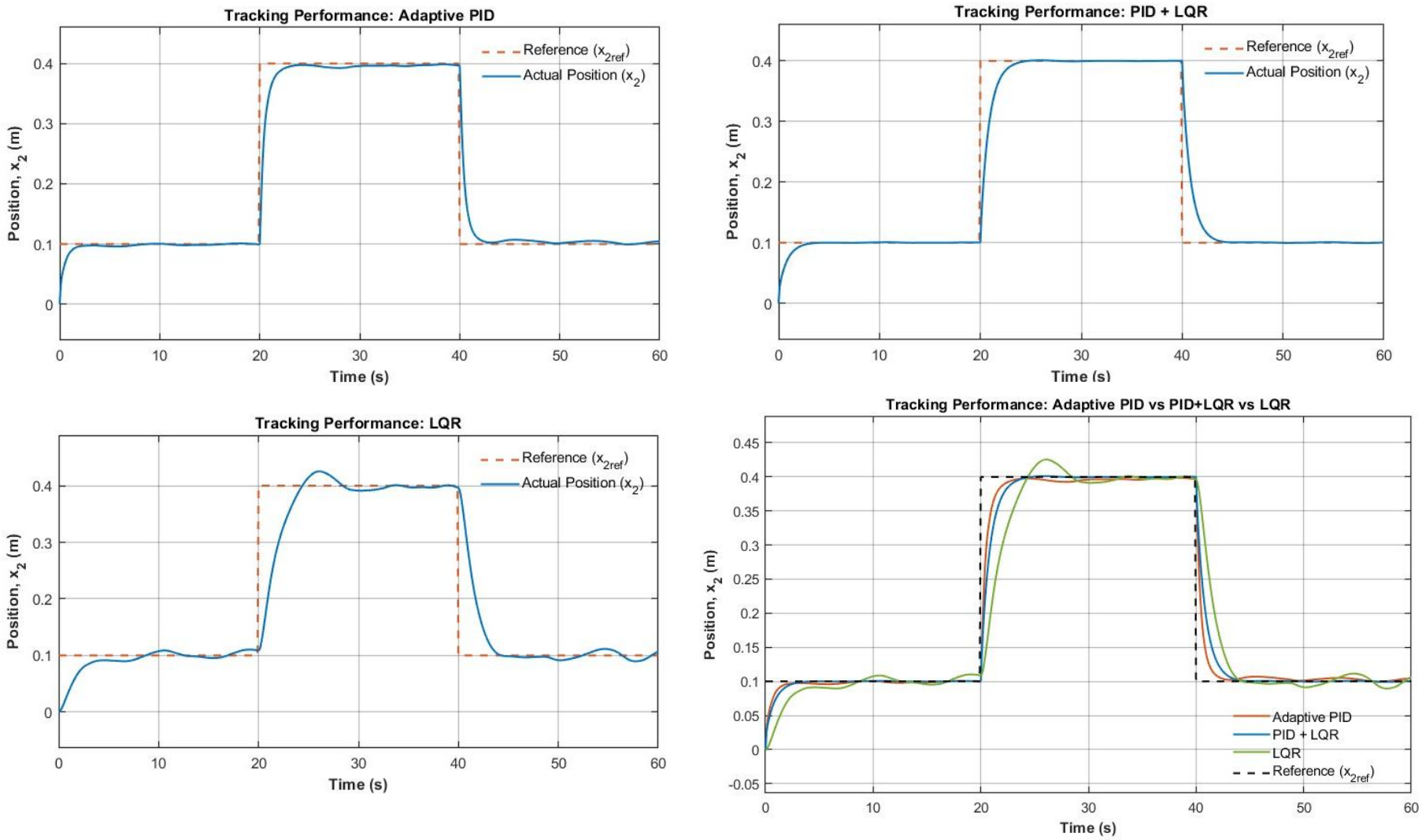


Figure 4.14: Tracking performance over time of Adaptive PID, PID+LQR and LQR

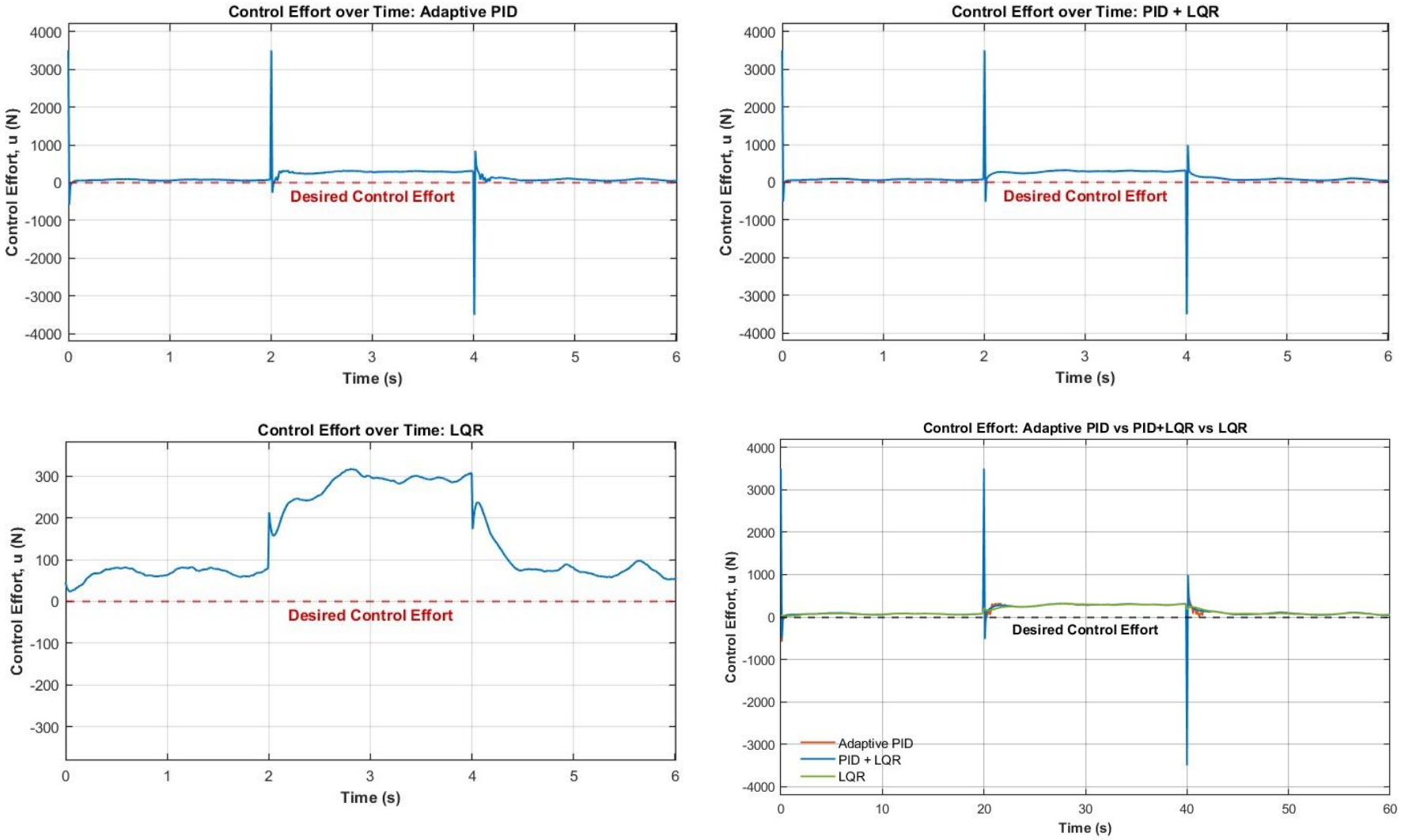


Figure 4.14: Energy usage over time of Adaptive PID, PID+LQR and LQR

V. The case for Reinforcement Learning

The control methods surveyed in preceding sections, from the classical PID controller to the sliding mode and model predictive frameworks, all share a foundational assumption: that the governing policy can be designed, whether by experience, by optimization, or by some combination of both, before the controller ever encounters the system it must govern. PID encodes this assumption in three fixed gains, tuned once and trusted thereafter. SMC encodes it in a sliding surface whose geometry is chosen to guarantee convergence under designed worst-case conditions, which are based on the assumptions and experience of the system designer. MPC encodes it in a receding-horizon optimization solved anew at every timestep, but whose cost function, constraints, and prediction model are all human-specified in advance; therefore, exceedingly prone to changes within the model. In each case, the engineer's knowledge of the system is the primary force of design, on which the algorithm merely executes.

Reinforcement Learning (RL) inverts this relationship. Rather than translating existing knowledge into a control law, an RL agent acquires knowledge by acting on the system and observing consequences. It begins blind and chaotic, with no model knowledge of the plant, no target trajectory, no notion of what constitutes good control. The RL Agent constructs its policy incrementally through millions of interactions, guided only by a scalar reward signal that encodes the engineer's objectives. The appeal is immediate: a sufficiently trained RL agent needs not be re-tuned when system parameters drift, when operating conditions change, or when the model used to design a classical controller turns out to have been wrong. It has learned the system's actual behavior, through millions of interactions and watt-hours spent learning the way a system reacts and acts upon its inputs.

In the context of the non-collocated coupled mass-spring-damper system studied here, this distinction is rather interesting. The noise injected onto the secondary mass, the nonlinear coupling between masses, and the rapidly changing reference profile together constitute an environment that exposes the limitations of fixed-gain design. A PID controller tuned for regulation performs poorly during aggressive transients. An SMC controller tuned for noise robustness and reduced chattering introduces wasted energy, or a motor prematurely worn, or a lax response time. An MPC controller optimized for tracking sacrifices robustness when its prediction model encounters unmodelled disturbances or the physical wear of a real system, while consuming a great amount of processing power for it to “foresee” the model. Each classical and modern method excels along one axis of the performance space while conceding ground on others — not because the methods are poorly designed, but because the performance demands are fundamentally in a tug-of-war, no single fixed algorithm can be optimal across all of them simultaneously.

This tug-of-war is also known as “**Pareto-optimal trade-off surface**” — the set of “best possible compromises” when one tries to optimize multiple conflicting parameters, improving one objective often worsens another.

It is directly from this tug-of-war that motivates the RL-augmented approach. The architecture proposed here — referred to throughout as **RL Meta-SMC** — does not replace the sliding mode controller. Instead, it positions a trained neural network as a supervisory layer above the SMC, responsible not for computing the control force directly but for continuously adapting the parameters that govern the SMC's behavior: the sliding surface coefficients, the switching gain, and the boundary layer thickness. The SMC retains its mathematical stability guarantees and its noise rejection properties. The neural network

contributes what the SMC cannot provide on its own: the situational awareness to recognize that a large reference step demands aggressive surface geometry, that proximity to the target demands a gentler boundary layer, and that sustained steady-state error demands a higher switching gain. What would eventually be acquired is a controller that inherits the robustness of sliding mode control and the adaptive foresight of a learned policy not too unlike that of a human brain. The structural integrity of modern control theory, overseen by the empiric intelligence of reinforcement learning.

"Why not just use Culture-Bee adaptive SMC or Adaptive PID?"

A natural objection to the proposed architecture is that the problem of fixed-gain algorithms' inadequacy is not unfamiliar to control designers, and those who study control literature, even the average users of said algorithms have long offered a solution: "Why not change those parameters on the fly?"

Adaptive PID schemes — among them model-reference adaptive control, self-tuning regulators, and simpler gain-scheduled variants — adjust their parameters "online" in response to measured performance degradation, typically by minimizing tracking error criterion through "gradient descent or recursive least squares". Adaptive SMC extends this philosophy to the sliding mode framework, employing parameter update laws derived from Lyapunov stability arguments to drive the switching gain or surface coefficients toward values that guarantee convergence despite parametric uncertainty. Both families of methods are mature, well-understood, and supported by decades of industrial deployment.

The fundamental limitation of classical adaptive control is that its adaptation mechanism is **local and reactive**. A self-tuning PID adjusts its gains in response to error that has already occurred. An adaptive SMC update law drives η upward when the sliding condition is violated after the violation has happened. The "adaptation" is, by design, a response to failure rather than an anticipation of it.

In short, even though "adaptive" algorithms do provide meaningful advantages in tracking performance or energy efficiency when well-tuned, they lack the fundamental understanding of how the model operates. The tuning for these adaptive schemes gets exponentially more of a herculean task as the systems they control scale up in complexity. For the coupled mass-spring-damper system studied here, where the reference profile changes rapidly, where noise acts continuously on the secondary mass, and where the performance objectives span rise time, steady-state precision, and energy consumption simultaneously, reactive adaptation is structurally too slow. By the time a Lyapunov-based update law has detected that η is insufficient and assign it a new appropriate value, the transient has already passed — and the same update law will stress the actuator to its limits before the next reference change arrives.

There too exists a second limitation. Classical adaptive laws are derived to optimize **one objective** — typically a Lyapunov function that guarantees stability or a quadratic error criterion that penalizes tracking deviation. They have no native mechanism for trading off competing objectives such as rise time against overshoot, or tracking precision against actuator energy consumption. Gain scheduling can approximate this by precomputing different parameter sets for different operating regions, but it requires the engineer to define those regions explicitly, to populate them with tuned parameters, and to design interpolation logic between them. This is, in effect, **manual meta-control**: the engineer performing by hand exactly the supervisory reasoning that the neural network in the proposed architecture learns automatically from experience. Therefore, the resulting schedule is brittle to operating conditions that fall outside the pre-specified regions.

Reinforcement learning bypasses both limitations by design. The trained agent does not react to failure, for it has encountered millions of simulated failures during training and has already learned what adjustments this exact scenario would expect before they occur in deployment. It does not optimize a single criterion, the reward function encodes the full scope of performance objectives, and the agent learns the Pareto-optimal trade-off surface through experience. And it requires no manual region definition, no interpolation logic, and no re-derivation when operating conditions shift, the generalization therefore is learned and reinforced.

The proposed **RL Meta-SMC** architecture can therefore be understood not as a rejection of adaptive control, but as its continuation. The replacement of hand-derived, single-objective, reactive adaptation laws with a learned, multi-objective, anticipatory and supervisory policy, one that retains the guarantees of the underlying SMC while extending its adaptability beyond what classical update laws can provide.

VI. Training and Validation Environment

1. "Reward" and "Kill switch"

a. The Reward function

What is "Reward":

In reinforcement learning, the reward signal is the sole channel through which the agent receives feedback about the quality of its behavior. Unlike a cost function in optimal control — which first analyzed against a known model and then minimized — the reward function is evaluated empirically at every timestep and accumulated across an episode, that is to say by means of observation or experience rather than theory or pure logic.

The agent has no direct access to the function itself; it observes only its output, which is a scalar value, and must infer from millions of such observations which sequences of actions tend to produce larger cumulative returns. The design of the reward function is therefore one of the most consequential decisions in the entire training pipeline. A poorly shaped reward produces a policy that is optimal with respect to the wrong objective — an agent that earns high scores by exploiting unintended loopholes rather than solving the intended control problem. A well-shaped reward encodes the full multi-objective specification of the controller — tracking precision, transient behavior, actuator constraints, and system safety — in a form that produces a smooth, learnable gradient landscape across the agent's experience.

For the RL Meta-SMC architecture proposed here, the reward function operates at the agent's update rate of 50 Hz and evaluates seven distinct criteria derived from the control objectives established earlier in Section I. These criteria are not wholly independent: they are deliberately ordered by priority and scaled so that the dominant objective, which is tracking x_2 to its reference, always contributes the largest magnitude signal, while secondary objectives such as energy efficiency and x_1 constraint compliance act as continuous shapers that guide the agent toward the desired operating envelope without overwhelming the primary tracking reward.

Grading rubric:

#	Component	Condition	Value
—	Base survival	Always	± 0.2
1	Tracking penalty	Always	$-4.0 \times e^2$ ($e = \lvert x_{2ref} - x_2 \rvert$)
2	Parachute penalty	$\lvert e \rvert < 50$ mm **and** $\lvert v_2 \rvert > 0.10$ m/s	$-1.0 \times \lvert v_2 \rvert$
3	Target lock bonus	$\lvert e \rvert < 10$ mm	$+0.60$
3	Target lock bonus	$\lvert e \rvert < 30$ mm	$+0.30$
4	Progress bonus	Always	$+0.2 \times \exp(-5 \times x_{2,error}^2)$
5	x_1 soft barrier	$\lvert x_1 \rvert > 0.30$ m	$-1.6(\lvert x_1 \rvert - 0.30^2)$
5	x_1 centering	Always	$0.06 \times \exp(-10x_1^2)$
6	Terminal penalty	$\lvert x_1 \rvert > 0.65$ m	-2000.0
7	Reward floor	Always	-10.0

At perfect tracking with $x_1 = 0$, a single timestep yields approximately **+1.06** (base + lock bonus + full progress + full centering). At maximum sustained error with x_1 approaching the barrier, a single timestep yields **−10.0** (absolute minimum reward). The terminal penalty of −2000 is applied once at the moment of x_1 violation and is designed to exceed the maximum cumulative tracking reward achievable in any episode, making x_1 wall contact unconditionally unprofitable regardless of tracking performance on the way there.

The "parachute penalty" in component 2 activates exclusively in the approach zone, which is within 50 mm of the target, but only when velocity is still high. This penalizes the specific failure mode of fast approach followed by overshoot/undershoot, without penalizing high velocity during the legitimate rise phase when the agent is far from the reference.

The code:

```
function [reward, next_u_prev] = calc_reward(x1, x2, x2_ref, u, ...
                                  time_since_change, v1, v2, u_prev)
% calc_reward.m — Phase 1: 50 Hz update rate
% Smooth tracking with scaled continuous rewards.

    reward      = double(0.2);              % Base survival reward
    next_u_prev = double(u);
    x2_error    = double(abs(x2_ref - x2));
    delta_u     = double(abs(u - u_prev));
    x1_abs      = double(abs(x1));

    % 1. Tracking — quadratic penalty on position error
    reward = double(reward - (4.0 * x2_error^2));

    % 2. Parachute — penalise high velocity in the approach zone
    if (x2_error < 0.050) && (abs(v2) > 0.10)
        reward = double(reward - (1.0 * abs(v2)));
    end

    % 3. Target lock bonus
    if x2_error < 0.010
        reward = double(reward + 0.6);      % Inside 10 mm
    elseif x2_error < 0.030
        reward = double(reward + 0.3);      % Inside 30 mm
    end

    % 4. Continuous progress bonus — Gaussian, fades with distance
    progress_bonus = double(0.2 * exp(-5.0 * x2_error^2));
    reward = double(reward + progress_bonus);

    % 5. x1 soft barrier and centering bonus
    if x1_abs > 0.30
        x1_margin1 = double(x1_abs - 0.30);
        reward     = double(reward - (1.6 * x1_margin1^2));
    end
    x1_centre_bonus = double(0.06 * exp(-10.0 * x1^2));
    reward = double(reward + x1_centre_bonus);

    % 6. Terminal penalty — x1 hard wall violation
    if x1_abs > 0.65
        reward = double(reward - 2000.0);
    end

    % 7. Per-step hard floor — prevents gradient-destroying spikes
    reward = double(max(reward, -10.0));

end
```

b. The Kill switch

What is the "Kill switch":

While the reward function shapes the agent's behavior through incentive, it alone cannot prevent a randomly-initialized or mid-training agent from producing physically destructive or computationally wasteful episodes. During early training especially, the agent may apply sustained incorrect gains due to the high noise-slow decay setting that drive the system into unbounded oscillation, saturate the actuator indefinitely, or simply fail to make any progress toward the reference for thousands of steps of simulated time. Should this be left unchecked, such episodes consume training time without contributing useful gradient information, and in the worst case, produce numerical instability in the Simulink solver that terminates the entire training run.

The kill switch — implemented as the input `is_done` signal for the RL Agent block — terminates an episode immediately when any of three conditions are met. It is not a penalty; it is a hard boundary on the episode's duration, analogous to the safety interlock on a physical test rig. The agent receives no reward for the steps it did not take after termination, which means premature death is extremely costly by definition, the agent loses all possible reward it would have earned had it survived. This creates a "natural" pressure toward survival without requiring an explicit survival bonus in the reward function, complementing the reward shaping described above.

The kill switch operates at the same 50 Hz rate as the agent and maintains four persistent state variables across timesteps: a loss-of-control timer, time elapsed since the last reference change, the magnitude of the last reference step, and the previous reference value. Each implemented as a unit delay in Simulink.

Kill switch conditions:

#	Condition	Trigger
1	Hardware limit	$\lvert x_1 \rvert > 0.65\ m$
2	Total loss of control	$\lvert x_{2,error} \rvert > 0.50\ m$ for $> 3.0s$
3	Settle time violation	$\lvert x_{2,error} \rvert > 30\ mm$ for $> 2.5s$ after last reference change

Condition A is unconditional and immediate. The 0.65 m threshold provides 150 mm of margin beyond the ±0.50 m constraint on x_1, sufficient to accommodate transient overshoot in the coupled mass dynamics without triggering premature termination during aggressive reference changes.

Condition B targets sustained divergence, a controller that has lost the plant entirely rather than one that is mid-transient. The 0.50 m error threshold is deliberate and is extremely wide for training purposes; the reference's step command routinely hit 0.30 m during the rise/fall phases and while overshoots/undershoots are allowed, it must not trigger this condition.

Condition C enforces the core tracking specification. The reference profile in this work changes every 3.0 seconds; a 2.5s settle-time budget therefore leaves 0.5s of margin between the required settle and the

next commanded step. The 30 mm error band matches the inner boundary of the target lock bonus in the reward function, ensuring that the survival criterion and the reward criterion are aligned.

The code:

```
function [is_done, next_timer_03, next_time_since_change, ...
          next_step_size, next_ref_prev] = check_safety( ...
          x1, x2, x2_ref, timer_03, time_since_change, ...
          current_step_size, x2_ref_prev)
% check_safety.m — Phase 1: 50 Hz
% Settle-time enforcement and hardware protection.

    is_done                = double(0);
    next_timer_03          = double(timer_03);
    next_time_since_change = double(time_since_change);
    next_step_size         = double(current_step_size);
    next_ref_prev          = double(x2_ref_prev);
    dt       = double(0.02);            % 50 Hz agent clock
    x2_error = double(x2_ref - x2);

    % Reference change detection
    if abs(x2_ref - x2_ref_prev) > 0.001
        next_time_since_change = double(0.0);
        next_step_size         = double(abs(x2_ref - x2_ref_prev));
    else
        next_time_since_change = double(time_since_change + dt);
    end
    next_ref_prev = double(x2_ref);

    % 1. hardware limit (x1 wall)
    if abs(x1) > 0.65
        is_done = double(1);
    end

    % 2. total loss of control
    if abs(x2_error) > 0.5
        next_timer_03 = double(timer_03 + dt);
    else
        next_timer_03 = double(0.0);
    end
    if next_timer_03 > 3.0
        is_done = double(1);
    end

    % 3. settle-time violation
    allowed_settle_time = double(2.5);
    error_band          = double(0.030);
    if (next_time_since_change > allowed_settle_time) && ...
       (abs(x2_error) > error_band)
        is_done = double(1);
    end

end
```

2. The Neural Network Training Script

a. Overview

The training script `NN_SMC_Agent_1.m` configures and runs the offline reinforcement learning process that produces the supervisory policy. It is structured in six sequential stages: environment registration, actor network construction, twin-critic network construction, hyperparameter configuration, training options, and the training call itself. The script is executed once — typically over several hours on a dedicated GPU, here, 12 hours on a 3080 Ti with 12GB VRAM and 64GB of tuned DDR5 system RAM — and produces a serialised agent file that is subsequently imported into the deployment Simulink model as a frozen inference block. After deployment, none of the training infrastructure is present at runtime; only the trained actor network's forward pass executes.

The observation vector carries full system state, reference context, the SMC's internal sliding surface value, the agent's previous action, and the magnitude of the most recent reference step. The bounds assigned to each element reflect empirically observed operating ranges of the physical system and are used by the TD3 critic to normalize its value estimates. The action vector represents the four SMC tuning parameters — C_2, C_4, η, and φ — each normalized to the interval $[-1, +1]$. Physical scaling to their respective operating ranges is handled by a dedicated MATLAB Function block in Simulink immediately downstream of the agent output port, decoupling the network's output convention from the SMC's parameter requirements. The output row order — $[C_2, C_4, \eta, \varphi]$ — is fixed by the declaration order of the action specification and is preserved exactly through the demux and scaling chain.

b. Architecture

What is TD3:

TD3 stands for **Twin Delayed Deep Deterministic Policy Gradient**. In the realm of Reinforcement Learning, if algorithms like DQN (Deep-Q Network) or DDPG (Deep Deterministic Policy Gradient) suffer from a common flaw of being too “optimistic”. They tend to overestimate how good a specific action is because they always take the maximum possible Q-value during updates. Over time, these small errors accumulate, leading to undesirable performance.

TD3 was introduced specifically to cure this overestimation bias. It introduces three major modifications to the DDPG architecture:

1. Clipped Double Q-Learning (The "Twin" Critics)

Instead of one Critic, TD3 uses two independent Critic networks. When evaluating an action, both Critics predict the future Q-value. The algorithm then takes the **minimum** of the two predictions to update the network.

This, by design, forces the AI to be pessimistic. It eliminates the overestimation bias because the Critics keep each other grounded.

2. Delayed Policy Updates

In standard algorithms, both the Actor and the Critic are updated on every single time step. But if the Critic is still inaccurate, updating the Actor is inefficient since the Critic is blind.

TD3 updates the Critic networks rapidly, but **delays** the Actor updates (typically updating the Actor only once for every two Critic updates). The Actor is only updated after the Critic has stabilized, resulting in a much smoother, more deliberate gradient descent.

3. Target Policy Smoothing

Neural networks can easily overfit to narrow peaks in the Q-function, exploiting sharp spikes that don't represent robust physical states. TD3 adds a small amount of clipped, random noise to the target action during training.

This forces the AI to evaluate the area around an action rather than a single point. This smooths out the value landscape, ensuring the policy is robust to slight mechanical disturbances.

c. Constructing the code

General architecture:

The actor network is a fully connected feedforward network mapping the 13-dimensional observation to the 4-dimensional action.

```
   [x1, x2, v1, v2, x2_ref, x2_error, time_since_change, s, C2_prev, C4_prev, ...
eta_prev, phi_prev, ref_jump] -> [C2_tune, C4_tune, Eta_tune, Phi_tune]
```

It comprises three hidden layers of 256 neurons each, separated by hyperbolic tangent activation functions, followed by a final linear layer and a terminal tanh activation that enforces the [−1, +1] output constraint at the network level.

Hyperparameters:

The TD3 agent is configured with a discount factor of $\gamma = 0.99$, enforced by the long-horizon nature of the tracking task, which dictates the agent must value smooth settling behavior several seconds into the future, not merely the immediate reward of the current timestep.

The experience replay buffer holds up to 10^6 transitions, sufficient to retain approximately 5.5 hours of experience at the 50 Hz agent clock, preventing the early-training bias that occurs when gradient updates draw exclusively from recent and suboptimal behaviors. Each gradient update samples a small batch of 256 transitions, a batch size chosen to saturate the GPU pipeline while providing stable gradient estimates as well as shorter training time. The critic learns at a rate of 3×10^{-4} and the actor at 1×10^{-4}, consistent with standard TD3 practice.

Exploration is governed by an additive Gaussian noise process. The initial variance of 1.0, applied to a [−1, +1] action space, drives the agent to explore the full physical range of each SMC parameter aggressively in early episodes. The variance decays at a rate of 5×10^{-6} per episode, which is slow enough that meaningful exploration persists well into the middle phase of the 5,000-episode curriculum, and is

floored at a minimum of 0.05 to prevent the policy from becoming fully deterministic. These settings are intentionally permissive for Phase 1: the goal is exploring coverages across the [C_2, C_4, η, φ] parameter space, not convergence to a single refined policy.

Training configuration and termination:

Each episode runs for a maximum of 3,000 steps, corresponding to 60 seconds of simulated time at the 50 Hz agent clock. At the 3.0s reference step interval defined by the training environment, this gives the agent 20 complete gates per episode, sufficient to observe the SMC's transient response, settling behavior, and steady-state performance across multiple distinct reference changes within a single experience trajectory. The kill switch described in Section V.1b terminates episodes early if any of the three safety conditions are triggered; the agent therefore receives variable-length episodes in practice, with early-training episodes typically terminating within the first few gates and mid-to-late-training episodes approaching the full 60-second duration.

The code:

```
clc; close all;

% ========================================================================
% CMSD Meta-RL Tuning Agent
% Architecture: Supervisory TD3 (0.02s Sample Time)
% Output: [C2, C4, Eta, Phi]
% ========================================================================

% === 1. DEFINE SIMULINK ENVIRONMEN) ===
Ts_AI = 0.02;  % (2x slower than the physics engine)
mdl   = 'RL_SMC_Training_Env';
open_system(mdl);

agentBlock = [mdl, '/RL Agent'];

% Observation: 13x1 Vector
% [x1, x2, v1, v2, x2_ref, x2_error, time_since_change, s, C2_prev, C4_prev,
eta_prev, phi_prev, ref_jump]
obsInfo = rlNumericSpec([13 1], 'Name', 'Observations');

% --- Limits for the input observation ---
% x1, x2:    Capped at +/- 0.8 (Death wall is at 0.65)
% v1, v2:    Capped at +/- 2.5 (Max observed is ~2.0)
% x2_ref:    Capped at +/- 0.6 (Saturation is -0.2 to 0.5)
% x2_error:  Capped at +/- 1.5 (Max possible difference)
% time:      Capped at 3.5s (Resets every 3.0s)
% s:         Capped at +/- 30 (Lower speeds mean smaller sliding surface)
% actions:   Must remain [-1, 1] for TD3 outputs
% ref_jump:  Capped at +/- 0.4 (Max jump is 0.3)

obsInfo.LowerLimit = [-0.8; -0.8; -2.5; -2.5; -0.6; -1.5;   0; -30; -1; -1; -1; -
1; -0.4];
obsInfo.UpperLimit = [ 0.8;  0.8;  2.5;  2.5;  0.6;  1.5; 3.5;  30;  1;  1;  1;
1;  0.4];

% Action: 4x1 Vector
```

```
    % [C2_tune, C4_tune, Eta_tune, Phi_tune]
    actInfo = rlNumericSpec([4 1], 'LowerLimit', -1, 'UpperLimit', 1, 'Name',
'SMC_Params');

    env = rlSimulinkEnv(mdl, agentBlock, obsInfo, actInfo);
    env.ResetFcn = @myResetFunction;

    % ==========================================================================
    % === 2. ACTOR NETWORK (SUPERVISOR) ===
    % ==========================================================================
    actorNetwork = [
        featureInputLayer(obsInfo.Dimension(1), 'Name', 'obs_in')
        fullyConnectedLayer(256, 'Name', 'fc1')
        tanhLayer('Name', 'tanh1')
        fullyConnectedLayer(256, 'Name', 'fc2')
        tanhLayer('Name', 'tanh2')
        fullyConnectedLayer(256, 'Name', 'fc3')
        tanhLayer('Name', 'tanh3')
        fullyConnectedLayer(actInfo.Dimension(1), 'Name', 'fc_out')
        tanhLayer('Name', 'tanh_limit')
    ];

    actor = rlContinuousDeterministicActor(actorNetwork, obsInfo, actInfo,
'UseDevice', 'gpu');

    % ==========================================================================
    % === 3. CRITIC NETWORKS ===
    % ==========================================================================
    criticNet1 = buildCriticGraph(obsInfo.Dimension(1), actInfo.Dimension(1));
    criticNet2 = buildCriticGraph(obsInfo.Dimension(1), actInfo.Dimension(1));

    critic1 = rlQValueFunction(criticNet1, obsInfo, actInfo, ...
        'ObservationInputNames', 'obs_in', 'ActionInputNames', 'action_out',
'UseDevice', 'gpu');

    critic2 = rlQValueFunction(criticNet2, obsInfo, actInfo, ...
        'ObservationInputNames', 'obs_in', 'ActionInputNames', 'action_out',
'UseDevice', 'gpu');

    % ==========================================================================
    % === 4. AGENT HYPERPARAMETERS ===
    % ==========================================================================
    criticOpts = rlOptimizerOptions('LearnRate', 3e-4, 'GradientThreshold', 1);
    actorOpts  = rlOptimizerOptions('LearnRate', 1e-4, 'GradientThreshold', 1);

    agentOptions = rlTD3AgentOptions;
    agentOptions.SampleTime               = Ts_AI;  % AI operates every 0.02s
    agentOptions.DiscountFactor           = 0.99;
    agentOptions.MiniBatchSize            = 256;
    agentOptions.ExperienceBufferLength   = 1e6;
    agentOptions.ActorOptimizerOptions    = actorOpts;
    agentOptions.CriticOptimizerOptions   = criticOpts;

    % High exploration variance to map out the tuning space quickly
    agentOptions.ExplorationModel.Variance          = 1.0;
```

```
    agentOptions.ExplorationModel.VarianceDecayRate = 5e-6;
    agentOptions.ExplorationModel.VarianceMin       = 0.05;

    % Create the new Meta Object
    metaObj = rlTD3Agent(actor, [critic1 critic2], agentOptions);

    agentDir = fullfile(pwd, 'Agents');
    if ~exist(agentDir, 'dir')
        mkdir(agentDir);
    end

    % =========================================================================
    % === 5. TRAINING OPTIONS ===
    % =========================================================================
    trainOpts = rlTrainingOptions(...
        'MaxEpisodes',              5000, ...
        'MaxStepsPerEpisode',       3000,   ...       % 3000 steps * 0.02s = 60
seconds
        'ScoreAveragingWindowLength', 50,  ...
        'Verbose',                  false, ...
        'Plots',                    'training-progress', ...
        'UseParallel',              false, ...
        'SaveAgentCriteria',        'AverageReward', ...
        'SaveAgentValue',           1700,   ...
        'StopTrainingCriteria',     'AverageReward', ...
        'StopTrainingValue',        2600,    ...
        'SaveAgentDirectory',       agentDir);

    % =========================================================================
    % === 6. START TRAINING ===
    % =========================================================================
    disp('Starting Meta-RL Training — GPU active.');
    trainingStats = train(metaObj, env, trainOpts);

    disp('Training complete.');

    figure('Name', 'Meta-RL Survival Performance', 'Color', 'w');
    plot(trainingStats.EpisodeIndex, trainingStats.EpisodeSteps, 'b-', 'LineWidth',
1.2);
    xlabel('Episode'); ylabel('Steps survived (Max 600)');
    title('Steps Survived Before Safety Kill');
    grid on;

    figure('Name', 'Average Reward', 'Color', 'w');
    plot(trainingStats.EpisodeIndex, trainingStats.AverageReward, 'r-', 'LineWidth',
1.5);
    xlabel('Episode'); ylabel('Average reward (50-ep window)');
    title('Training Convergence');
    yline(0, '--k', 'Zero line'); grid on;


    % =========================================================================
    % === CUSTOM MATLAB FUNCTIONS ===
    % =========================================================================
```

```
    % 1. Twin-Critic Graph Generator
    function criticNet = buildCriticGraph(obsDim, actDim)
        obsPath = [
            featureInputLayer(obsDim,  'Name', 'obs_in')
            fullyConnectedLayer(256,   'Name', 'obs_fc')
            tanhLayer('Name', 'obs_tanh')
        ];
        actPath = [
            featureInputLayer(actDim,  'Name', 'action_out')
            fullyConnectedLayer(256,   'Name', 'act_fc')
            tanhLayer('Name', 'act_tanh')
        ];
        commonPath = [
            additionLayer(2,           'Name', 'add')
            tanhLayer('Name', 'relu_1')
            fullyConnectedLayer(256,   'Name', 'fc_2')
            tanhLayer('Name', 'relu_2')
            fullyConnectedLayer(1,     'Name', 'QValue')
        ];
        criticNet = layerGraph(obsPath);
        criticNet = addLayers(criticNet, actPath);
        criticNet = addLayers(criticNet, commonPath);
        criticNet = connectLayers(criticNet, 'obs_tanh',  'add/in1');
        criticNet = connectLayers(criticNet, 'act_tanh',  'add/in2');
    end

    % 2. Seed randomizer
    function in = myResetFunction(in)
        new_noise = randi([1, 100000]);
        new_ref   = randi([1, 100000]);

        in = setVariable(in, 'noise_seed', new_noise);
        in = setVariable(in, 'ref_seed',   new_ref);

        %fprintf('Episode Reset -> Noise Seed: %d | Ref Seed: %d\n', new_noise,
new_ref);
    end
```

3. Simulink Training Environment

a. The reference step generator

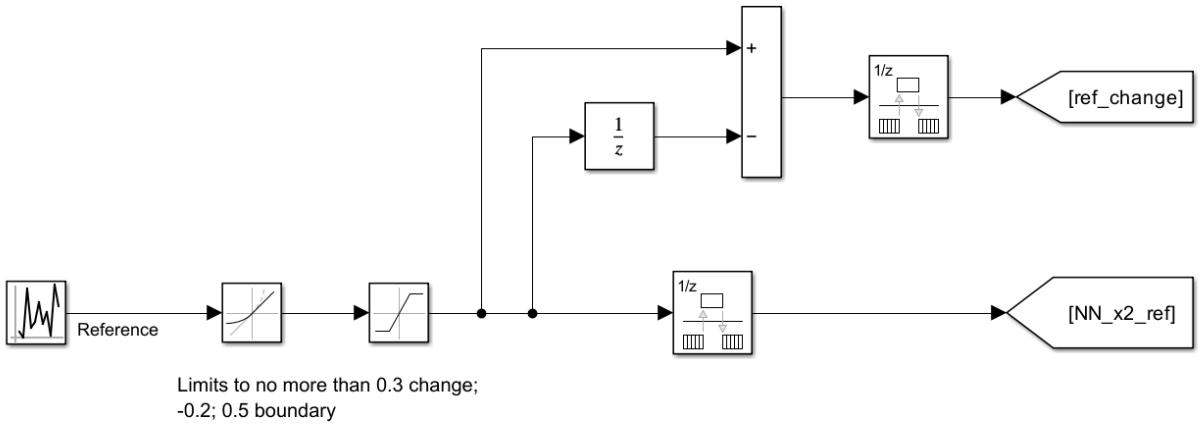


Figure 5.1: The reference step generator

The Uniform Random Number block is set to [-0.2; 0.5]. The signal is then passed onto the Rate Limiter block set at [-0.1; 0.1], with sampling rate of 3s, this effectively means between two steps there could not be more than 0.3m in difference. The Saturation block is set at [-0.2; 0.5] to ensure the end reference never exceeds this physical boundary.

b. The RL Agent block and the Translator

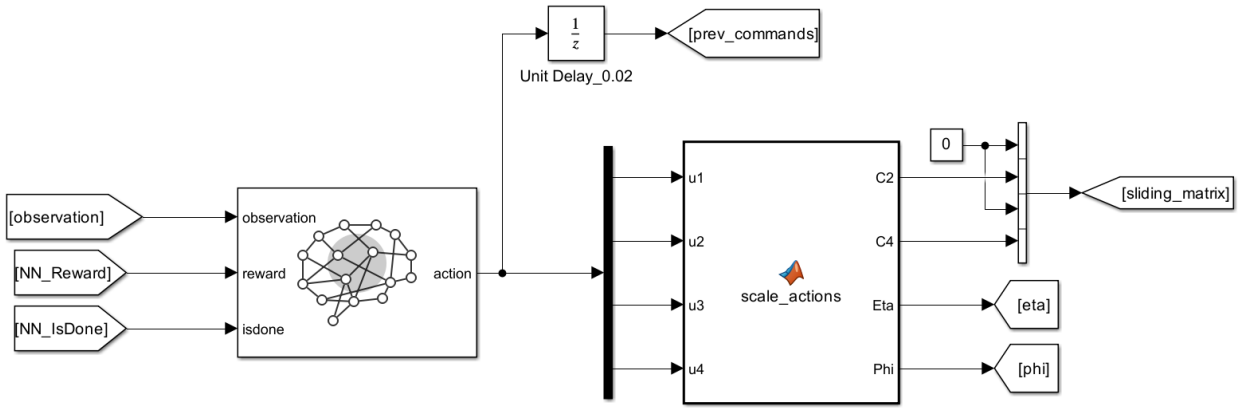


Figure 5.2: The RL Agent and its scaler

The `scale_action` block's only function is to translate the Agent's [-1; 1] command outputs into a set of parameters from which the SMC could "supervised" and "acted upon". The Range of these parameters had been empirically tested to produce a desirable result especially with the Uniform Random Number's seed being set to `12345`.

```
% --- 1. C2: Speed vs. Overshoot ---
% Baseline: 20.0 | Ceiling: 25.0 | Range: [15.0 to 25.0]
% If u1 = 1, C2 = 25. If u1 = -1, C2 = 15.
C2 = double(20.0 + (5.0 * u_1));

% --- 2. C4: Damping ---
% Baseline: 2.0 | Scale: 1.0 | Range: [1.0 to 3.0]
C4 = double(2.0 + (1.0 * u_2));

% --- 3. Eta: Reaching Law (Steady-State Error) ---
% Center: 100.0 | Scale: 50.0 | Range: [50.0 to 150.0]
Eta = double(100.0 + (50.0 * u_3));

% --- 4. Phi: Boundary Layer ---
% Center: 0.100 | Scale: 0.070 | Range: [0.030 to 0.170]
% Ensures Phi never drops below the 0.027 chatter threshold.
Phi = double(0.100 + (0.070 * u_4));
```

c. The SMC and the Plant

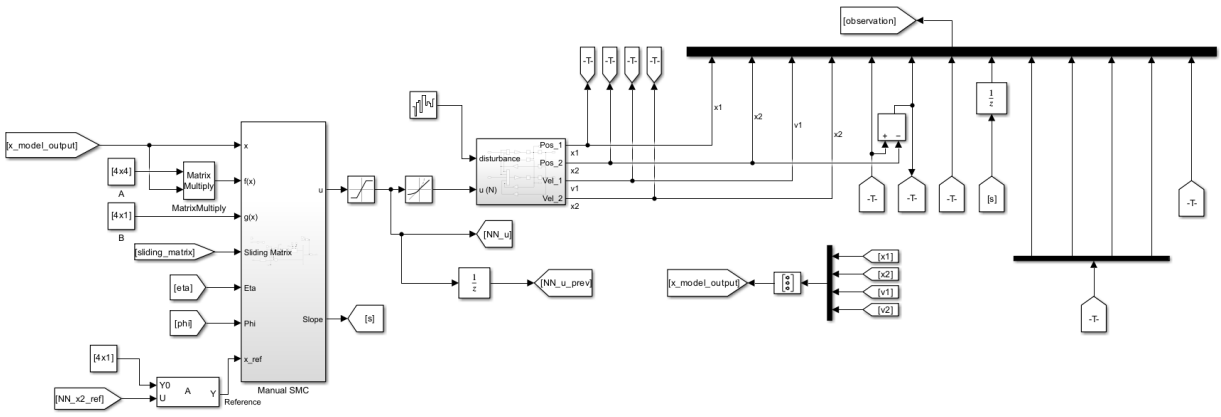


Figure 5.3: SMC and the Plant as well as the 13D Mux for the Observation vector

The SMC and the plant are virtually unchanged compared to Section II's breakdown diagrams. The only significant change is that the Controller had been made to accept its sliding surface C, its reaching rate η, and boundary φ layer as variables.

The `scale_action` block's only function is to translate the Agent's [-1; 1] command outputs into a set of parameters from which the SMC could "supervised" and "acted upon".

d. The Kill switch and Reward block

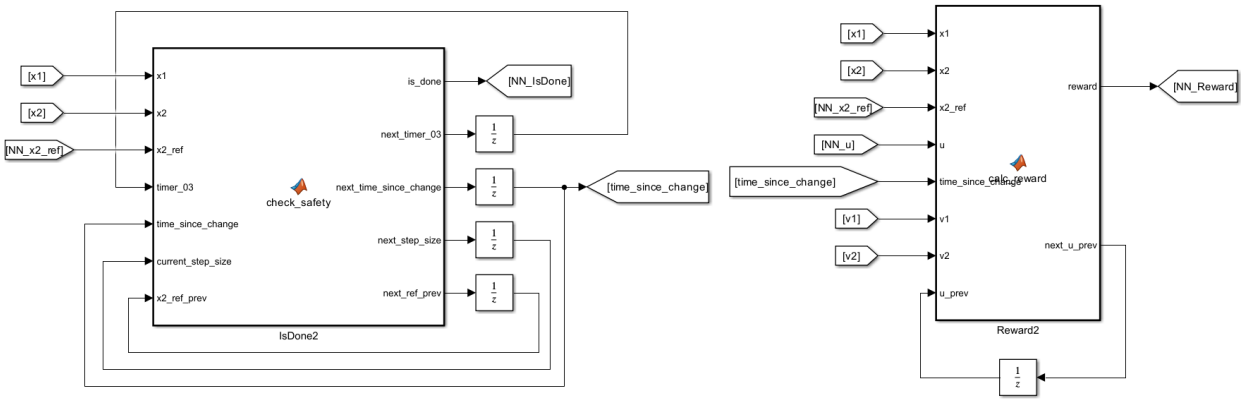


Figure 5.4: The Kill switch and Reward blocks with Unit Delays

4. Training and Benchmarking against other Agents

a. Training and After training

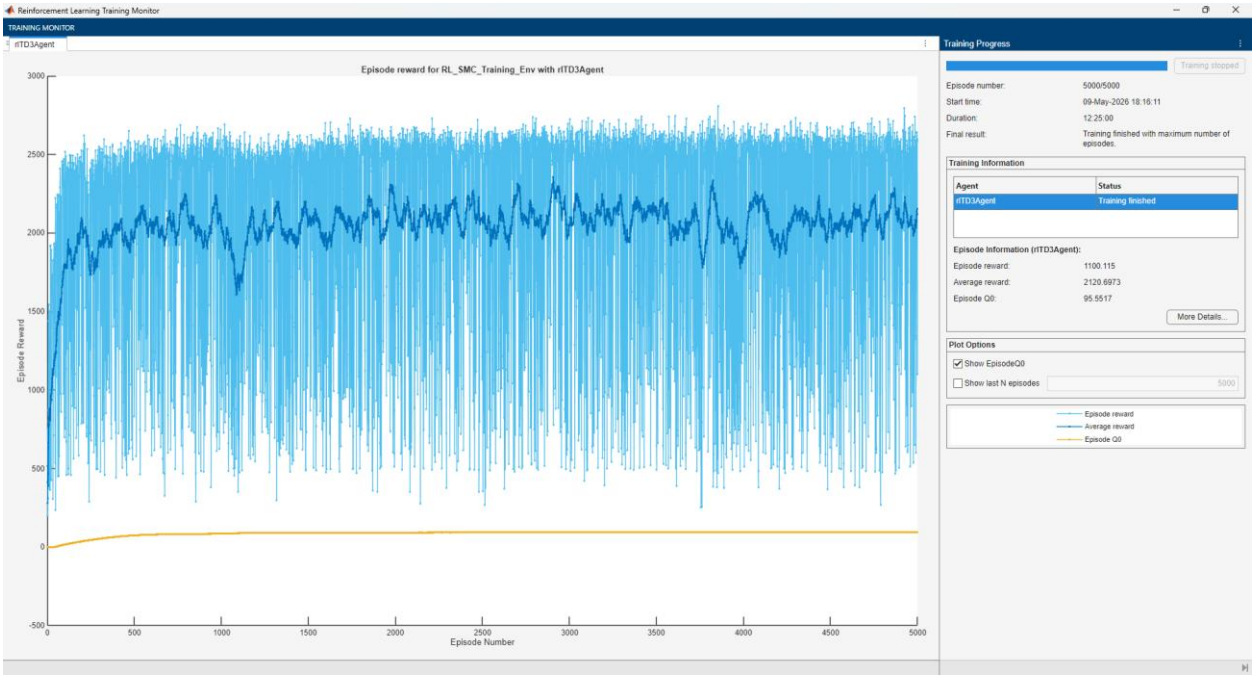


Figure 5.5: Episode reward and Average 50-episode reward after 5000 episodes/12 hours

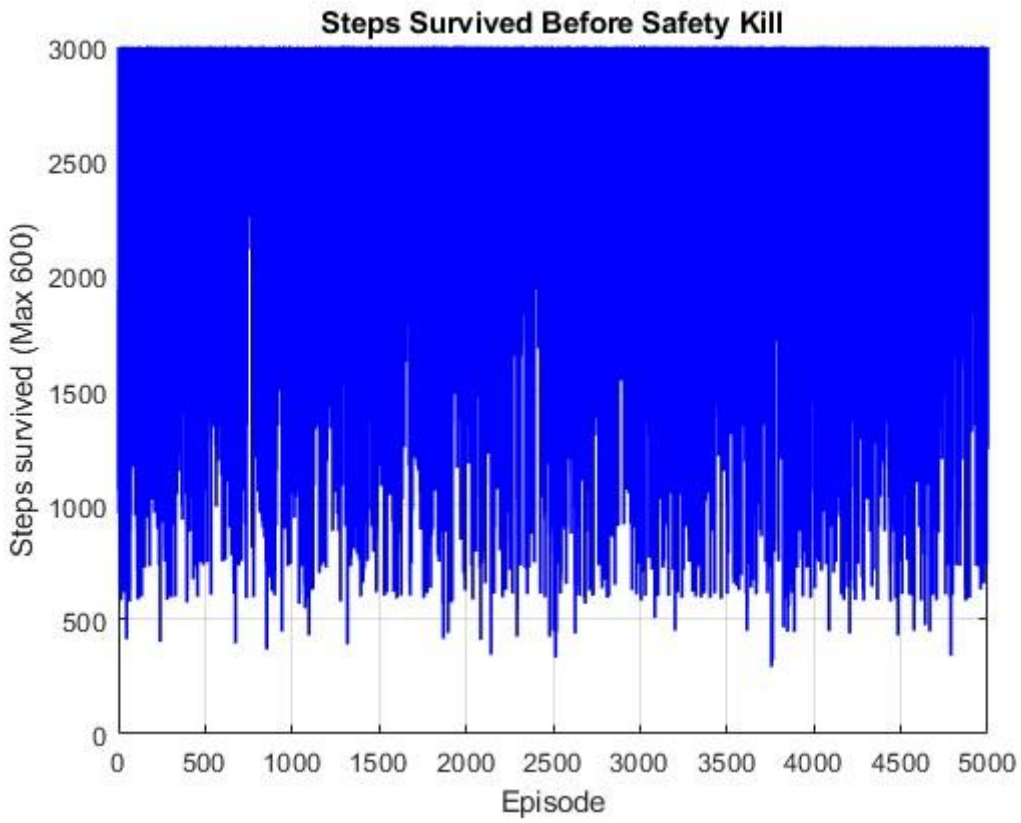


Figure 5.6: Steps survived of each episode

We can see that most of the episodes start to converge on an optimal plan to survive around episode 1200, which is when the average 50-episode reward stops showing meaningful increases, but rather the peaks and troughs of the training script's high variance and slow decay rate.

b. Benchmarking against other Agents

With this, we add another term to the Reward function, not to retrain the Agents but to "benchmark" all the agents based on `'noise_seed' = 23341; 'ref_seed' = 12345`. This term will find an Agent that has the least chattering of *u*, after which it will be benchmarked against the final 5000th Agent and against the fixed-parameter SMC.

The term is as follows:

```
% --- CHATTER REDUCTION ---
% The max possible sum is 8.0. If the AI moves its joysticks more than
% a combined 0.2 (10% total shift) in a single 0.02s frame, will be penalized.
if delta_u_sum > 0.2
    reward = double(reward - (0.05 * delta_u_sum));
end
```

The benchmark's code:

```
clc;

% dummy seeds
assignin('base', 'noise_seed', 23341);
assignin('base', 'ref_seed', 12345);

fprintf('Noise Seed: 23341 | Ref Seed: 12345\n');

% --- 1. Rebuild env
mdl = 'RL_SMC_Training_Env';
open_system(mdl);
agentBlock = [mdl, '/RL Agent'];

% Rebuild 13x1 limits
obsInfo = rlNumericSpec([13 1], 'Name', 'Observations');
obsInfo.LowerLimit = [-0.8; -0.8; -2.5; -2.5; -0.6; -1.5;   0; -30; -1; -1; -1; -
1; -0.4];
obsInfo.UpperLimit = [ 0.8;  0.8;  2.5;  2.5;  0.6;  1.5; 3.5;  30;  1;  1;  1;
1;  0.4];
actInfo = rlNumericSpec([4 1], 'LowerLimit', -1, 'UpperLimit', 1, 'Name',
'SMC_Params');

env = rlSimulinkEnv(mdl, agentBlock, obsInfo, actInfo);

% --- 2. Setup
agentDir   = fullfile(pwd, 'Agents');
renamedDir = fullfile(pwd, 'Renamed_Agents');

if ~exist(renamedDir, 'dir')
    mkdir(renamedDir);
    disp('Created \Renamed_Agents folder.');
end

files = dir(fullfile(agentDir, '*.mat'));
simOpts = rlSimulationOptions('MaxSteps', 3000, 'NumSimulations', 1);

disp(['Found ', num2str(length(files)), ' agents.']);
fprintf('-----------------------------------------------------\n');

% --- 3. loop
for i = 1:length(files)
    oldName = files(i).name;
    fprintf('Testing %s... ', oldName);

    try
        % DYNAMIC LOAD: Load the file as a struct, grab the first variable
        loadedData = load(fullfile(agentDir, oldName));
        varNames   = fieldnames(loadedData);
        test_agent = loadedData.(varNames{1}); % Extracts the brain

        experience = sim(env, test_agent, simOpts);

        % Calculate total exact score from the run
```

```
            epScore = sum(experience.Reward);
            fprintf('Score: %.2f\n', epScore);

            % Extract any numbers from the old name
            epNumStr = regexp(oldName, '\d+', 'match');
            if ~isempty(epNumStr)
                epNum = epNumStr{end};
            else
                epNum = num2str(i);
            end

            % Apply the new name and copy
            newName = sprintf('Tested_Reward%.0f_Agent%s.mat', epScore, epNum);
            copyfile(fullfile(agentDir, oldName), fullfile(renamedDir, newName));

        catch ME
            % If fails, print reason
            fprintf(2, 'FAILED! Reason: %s\n', ME.message);
        end
    end

    fprintf('-----------------------------------------------------\n');
    disp('Complete! Check \Renamed_Agents.');
```

After running the benchmark, Agent 3971 has the highest score of 629. And the last, Agent 5000, scored a total of 614. In the dictionary of controller design, a 15-score mismatch is hardly a landslide victory for Agent 3971, the preset `'noise_seed' = 23341; 'ref_seed' = 12345` simply favored Agent 3971's tracking algorithm and ignored Agent 5000's aggregated experience over the past 5000 training episodes and its massive 10^6 steps of memory.

c. The Monte Carlo Evaluation

What is a Monte Carlo Evaluation:

Monte Carlo Evaluation of Monte Carlo Methods are a broad class of computational algorithms based on repeated random sampling for obtaining numerical results. The underlying concept is to use randomness to solve deterministic problems.

Due to our purposes and limitations derived from hardware and time, our Monte Carlo test will be set to run 100 times each, on two Agents, 3971 and 5000 only.

Monte Carlo Evaluation code:

```
clc;
disp('Initializing the Monte Carlo Eval ...');

% --- 1. Generate 100 Deterministic Shared Seeds
numRuns = 100;
rng(44); % RNG number will be changed or locked
shared_ref_seeds   = randi([1, 100000], numRuns, 1);
shared_noise_seeds = randi([1, 100000], numRuns, 1);

% --- 2. Load the Two full Agents
agentDir = fullfile(pwd, 'Renamed_Agents');

disp('Loading Full Agent 3971...');

data3971 = load(fullfile(agentDir, 'Tested_Reward629_Agent3971.mat'));
varNames = fieldnames(data3971);
agent3971 = data3971.(varNames{1});

disp('Loading Full Agent 5000...');

data5000 = load(fullfile(agentDir, 'Tested_Reward614_Agent5000.mat'));
varNames = fieldnames(data5000);
agent5000 = data5000.(varNames{1});

% --- 3. Preallocate Score Arrays
scores_3971 = zeros(numRuns, 1);
scores_5000 = zeros(numRuns, 1);

mdl = 'RL_SMC_Training_Env';

% Setup parallel pool if available (Uses all CPU cores)
pool = gcp('nocreate');
if isempty(pool)
    disp('Spinning up parallel CPU workers...');
    parpool;
end

disp(['Starting ', num2str(numRuns), '-episode parallel gauntlet.']);
disp('Loading CPU cores...');
fprintf('-------------------------------------------------------\n');

% --- 4. Parallel procesisng

parfor i = 1:numRuns

    load_system(mdl);

    % Inject the shared seeds
    assignin('base', 'ref_seed', shared_ref_seeds(i));
    assignin('base', 'noise_seed', shared_noise_seeds(i));

    % Rebuild 13x1 limits
    obsInfo = rlNumericSpec([13 1], 'Name', 'Observations');
```

```
        obsInfo.LowerLimit = [-0.8; -0.8; -2.5; -2.5; -0.6; -1.5;   0; -30; -1; -1; -
1; -1; -0.4];
        obsInfo.UpperLimit = [ 0.8;  0.8;  2.5;  2.5;  0.6;  1.5; 3.5;  30;  1;  1;
1;  1;  0.4];
        actInfo = rlNumericSpec([4 1], 'LowerLimit', -1, 'UpperLimit', 1, 'Name',
'SMC_Params');

        % Create env
        agentBlock = [mdl, '/RL Agent'];
        env = rlSimulinkEnv(mdl, agentBlock, obsInfo, actInfo);

        % Strict 1-episode test configuration
        simOpts = rlSimulationOptions('MaxSteps', 3000, 'NumSimulations', 1);

        % ----- Evaluate Agent 3971 -----
        exp3971 = sim(env, agent3971, simOpts);
        scores_3971(i) = sum(exp3971.Reward);

        % ----- Evaluate Agent 5000 -----
        exp5000 = sim(env, agent5000, simOpts);
        scores_5000(i) = sum(exp5000.Reward);

    end

    % --- 5. Calculate and Display Results ---
    fprintf('-----------------------------------------------------\n');
    disp('Evaluation Complete!');
    fprintf('Agent 3971 Average Score: %.2f\n', mean(scores_3971));
    fprintf('Agent 5000 Average Score: %.2f\n', mean(scores_5000));

    % Determine the winner
    fprintf('-----------------------------------------------------\n');
    if mean(scores_3971) > mean(scores_5000)
        fprintf('WINNER: Agent 3971 (By %.2f points)\n', mean(scores_3971) -
mean(scores_5000));
    elseif mean(scores_5000) > mean(scores_3971)
        fprintf('WINNER: Agent 5000 (By %.2f points)\n', mean(scores_5000) -
mean(scores_3971));
    else
        disp('TIE');
    end
```

The Monte Carlo Evaluation is run three times with different RNG seeds (42, 16 and 44):

With RNG = 42:

```
Agent 3971 Average Score: 1094.57
Agent 5000 Average Score: 1007.53
```

With RNG = 16:

```
Agent 3971 Average Score: 1062.92
Agent 5000 Average Score: 990.99
```

With RNG = 44:

```
Agent 3971 Average Score: 1151.37
Agent 5000 Average Score: 1065.71
```

By surviving 300 distinct, randomized tracks across three different seeds and consistently pulling ahead by roughly 70 to 85 points, Agent 3971 is the statistically proven generalist.

This scenario highlights a phenomenon in deep reinforcement learning: **Policy Degradation**.

While it is enticing to assume that more training always equals a smarter brain, and that one could brute force a smarter brain by extending the training time or by regulating a harsher rewarding regime, TD3 networks can drift. Around episode 3971, the Actor network had converged upon an optimized global minimum for navigating momentum shifts of the Coupled Mass-Spring-Damper system. However, because training continued for another 1,000 episodes with active exploration noise, the network was forced to keep tweaking its weights. By episode 5000, the AI had slightly "blunted" its own edge, likely overwriting some of its optimal internal parameters with slightly more conservative ones purely by chance.

5. Benchmarking and Stability against fixed-parameters SMC

For this benchmark, the Normal SMC's parameters are as follows:

Sliding Coefficient Matrix: [0; 15; 0; 2]

Exponential Reaching Law: $\eta = 100; K = 10$

Hyperbolic Tangent boundary layer: $\phi = 0.1$

The White Noise Generator's seed is **23341**, while the Uniform Random Number's seed is **12345**.

Each Controller's performance is evaluated through these criteria:

Average energy usage.

Min and Max energy usage.

Average Tracking error.

All of which are calculated within Simulink itself through a series of blocks.

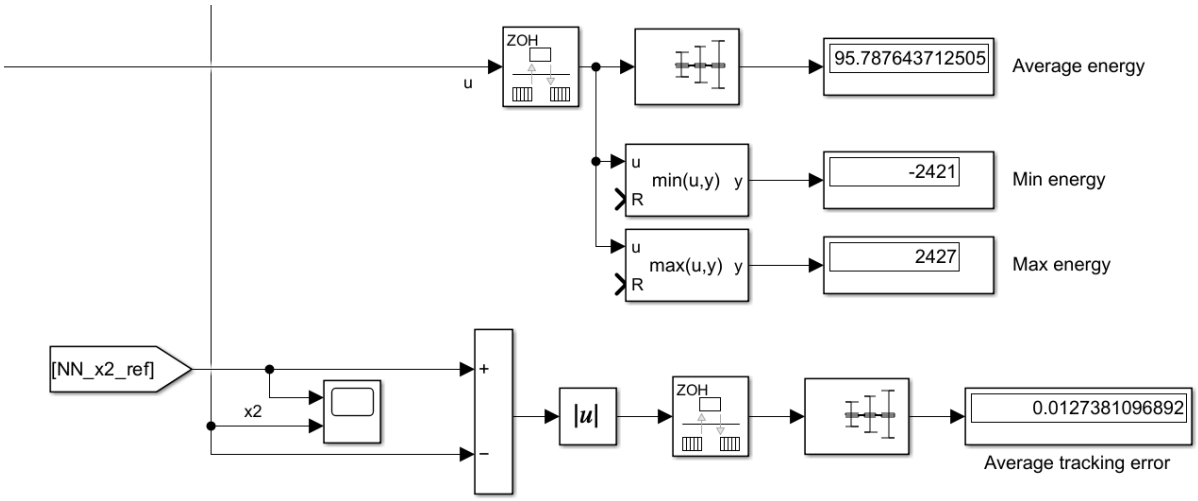


Figure 5.7: The "Grading blocks" in Simulink

a. Tracking performance

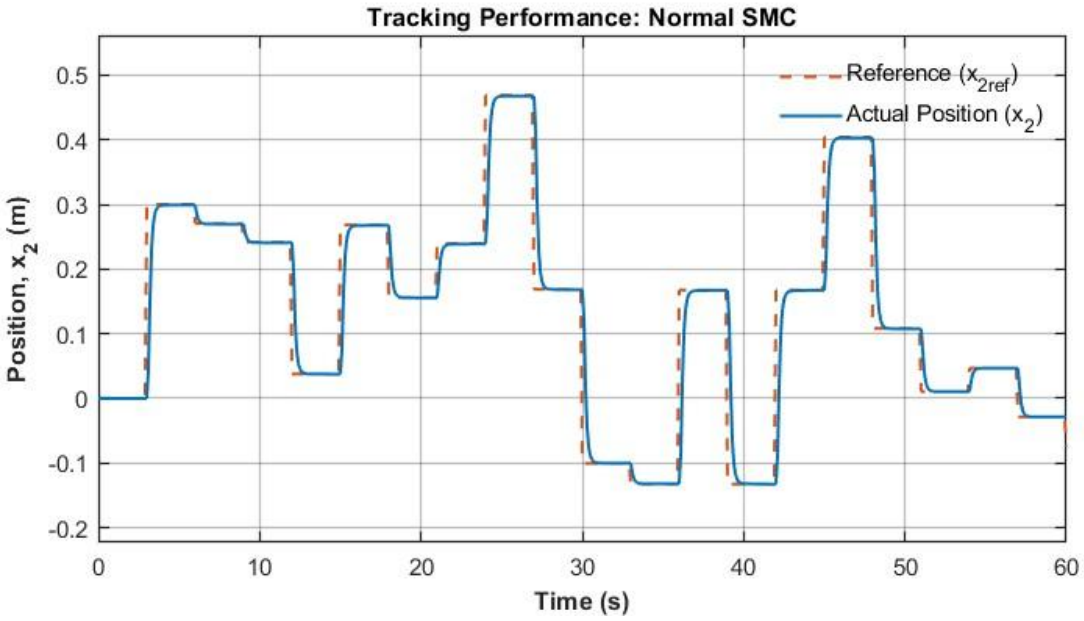


Figure 5.8: Normal SMC's tracking performance

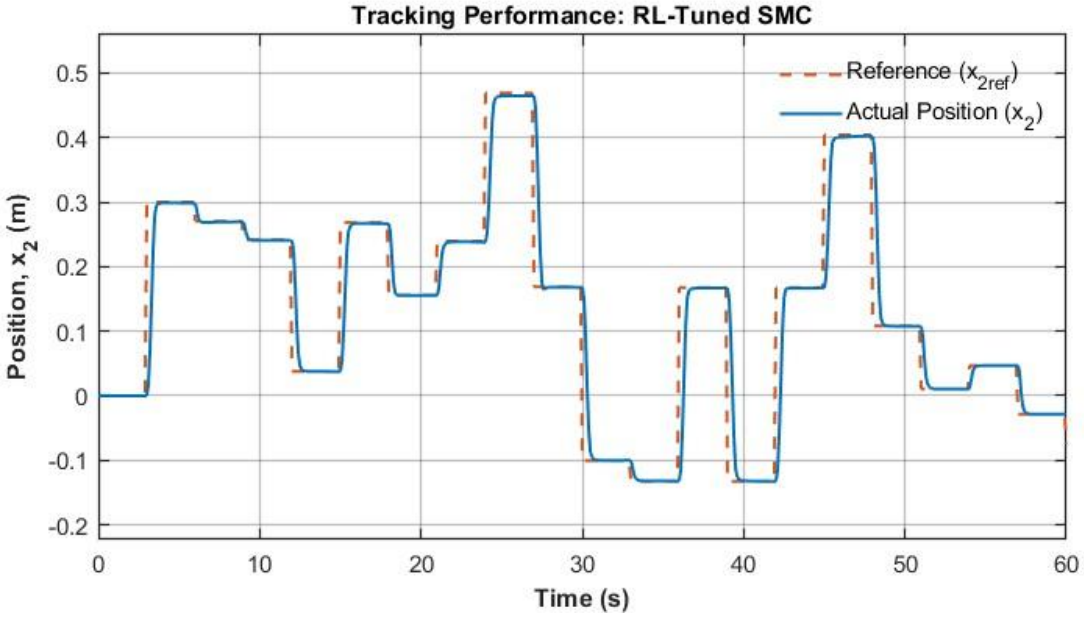


Figure 5.9: Meta-RL SMC's tracking performance

b. Energy efficiency and Motor wear

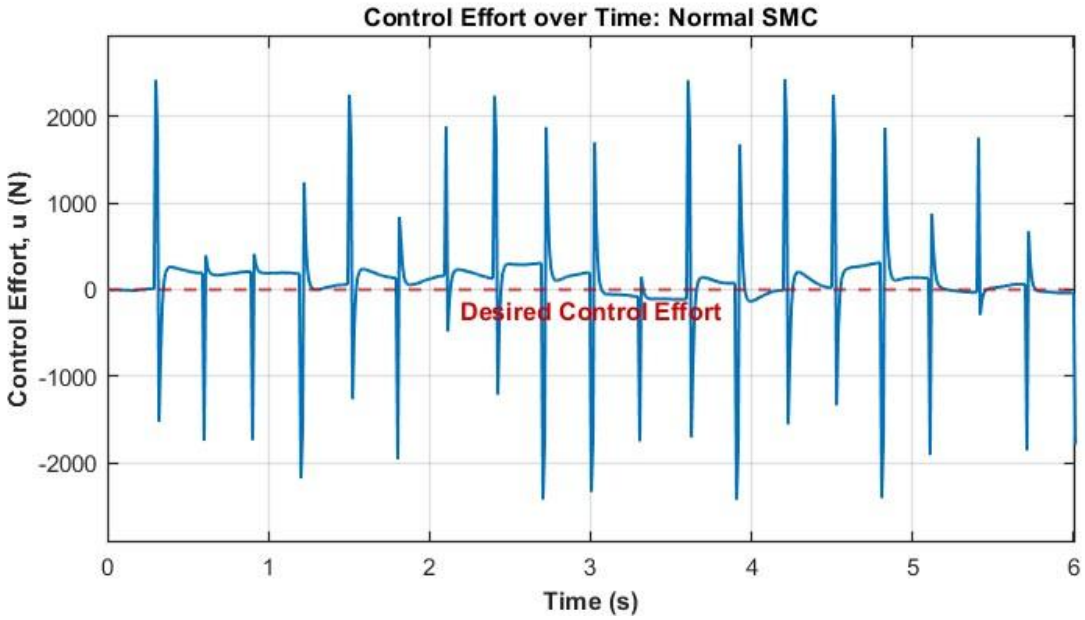


Figure 5.10: Normal SMC's energy usage

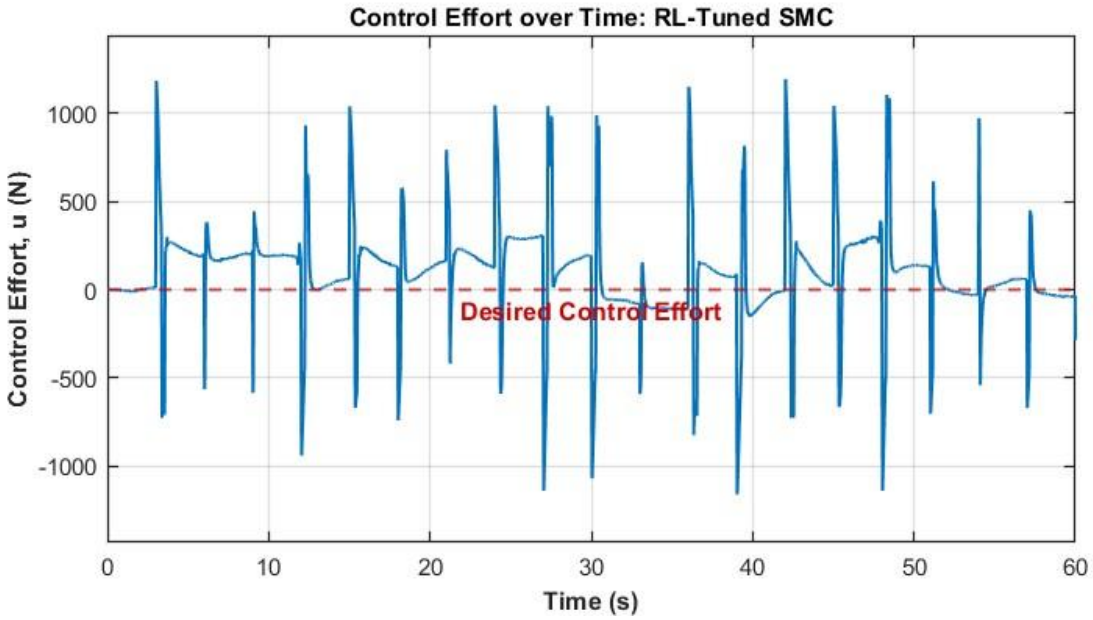


Figure 5.11: Meta-RL SMC's energy usage

c. Results and verdict

Normal SMC's performance:

Average Energy usage: 95.7876 N

Min Energy usage: -2421 N

Max Energy usage: 2427 N

Average tracking error: 0.0127 m

Meta-RL SMC's performance:

Average Energy usage: 96.0520 N

Min Energy usage: -1162 N

Max Energy usage: 1196 N

Average tracking error: 0.0169 m

Energy: The average usage is almost identical (95.8 vs 96.1 N), but the peak forces is tremendously different. The fixed SMC swings to around ±2,427 N while the Meta-RL SMC caps at ±1,196 N. That is roughly **half the peak actuator stress** (51% less peak energy) for the same average energy expenditure. The Meta-RL agent has learned to spread force more evenly rather than hammering the actuator in sharp bursts, that is when superimposing the Agent's *u* graph onto that of the SMC's, the Agent anticipated *and* successfully moderated its actuation slope to be less steep whenever there is a reference change, while the actuation curve remains roughly similar across the settling regimes of the test.

Tracking error: Fixed SMC wins at 12.7 mm vs 16.9 mm. The Meta-RL agent is 33% worse on average tracking. In our `calc_reward` function, we gave the AI a +0.3 target lock bonus as long as it stayed within a 30mm (0.030m) error margin. The Agent discovered it could earn almost the full progress bonus while accepting a slightly larger steady-state error, which in this case is a completely rational response to our reward landscape.

The "parachute penalty" $-1.0 \times |v_2|$ when inside 50 mm with high velocity actively discouraged fast aggressive approaches. Combined with the x_1 soft barrier discouraging large force excursions, the agent learned a conservative, smooth policy which explains the halved peak forces and the slightly larger steady-state error.

Verdict:

The Meta-RL SMC is not better overall — it has made a specific trade off that our reward function asked for. It is meaningfully better on one axis (peak force reduction: −51%) and meaningfully worse on another (tracking error: +33%). Whether that trade is acceptable depends on the system's real priorities.

This demonstrates that the RL agent had learned a genuine policy rather than just approximating the fixed SMC. The agent found a different point on the Pareto frontier between tracking precision and actuator stress, which is exactly what a meta-controller should be capable of.

Furthermore, the tracking gap is fixable. Increasing the tracking penalty coefficient from 4.0 to 8.0–10.0, or tightening the target lock bonus boundaries, would shift the reward landscape toward precision. The agent would likely close much of the 4.2 mm gap while still maintaining smoother actuation than the fixed SMC. Yet that should be reserved for a “Phase 2” approach to further train this Agent.

In short, we’ve built a brain that prioritizes hardware survival over mathematically unnecessary micro-precision (at least it had approximated as such due to our lax reward rubric).

RL Agent's adaptation:

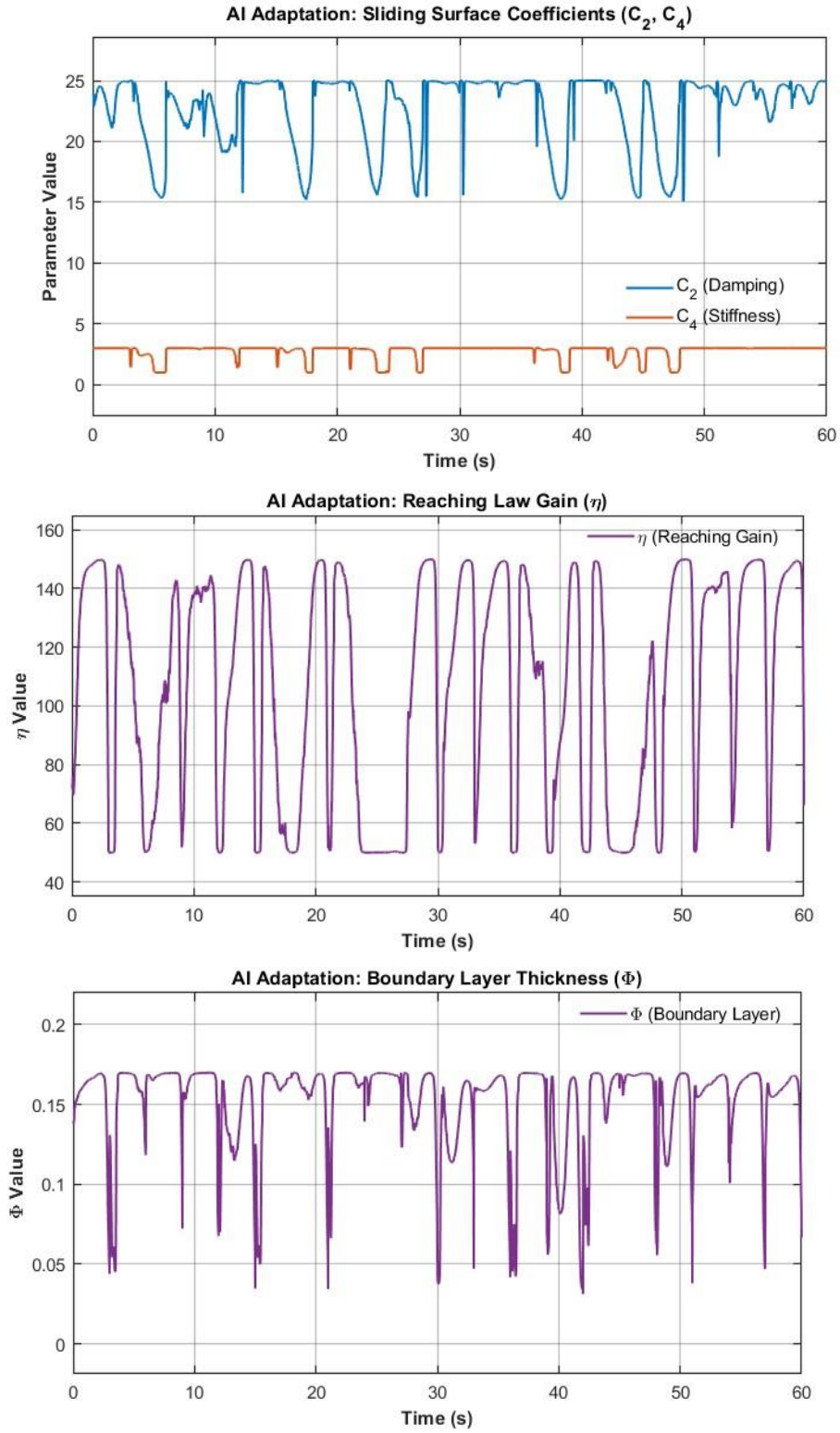


Figure 5.12: The RL Agent's dynamic [C_2, C_4, η, φ]

Comparison graphs:

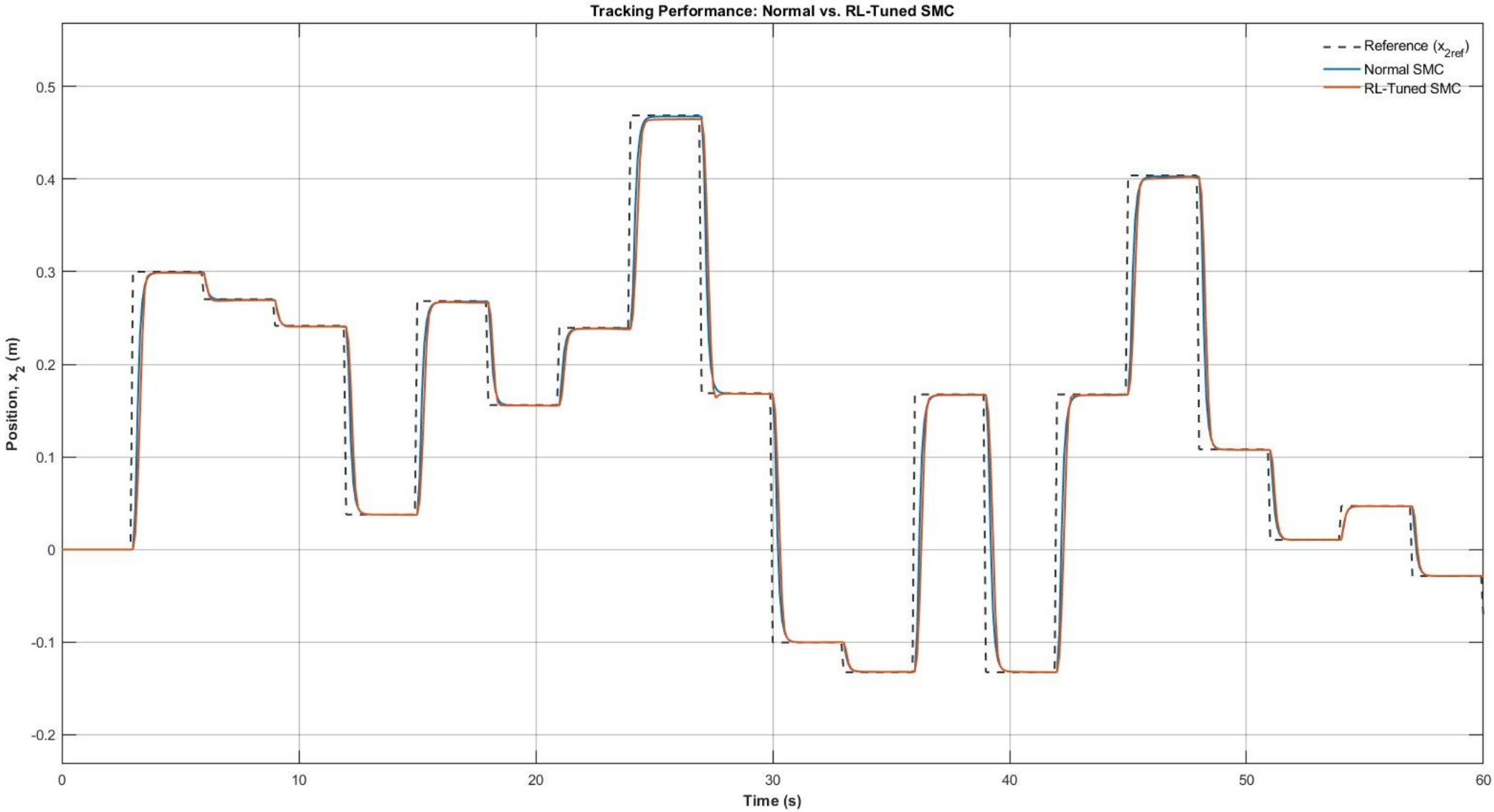


Figure 5.13: Meta-RL SMC vs Normal SMC's tracking performance

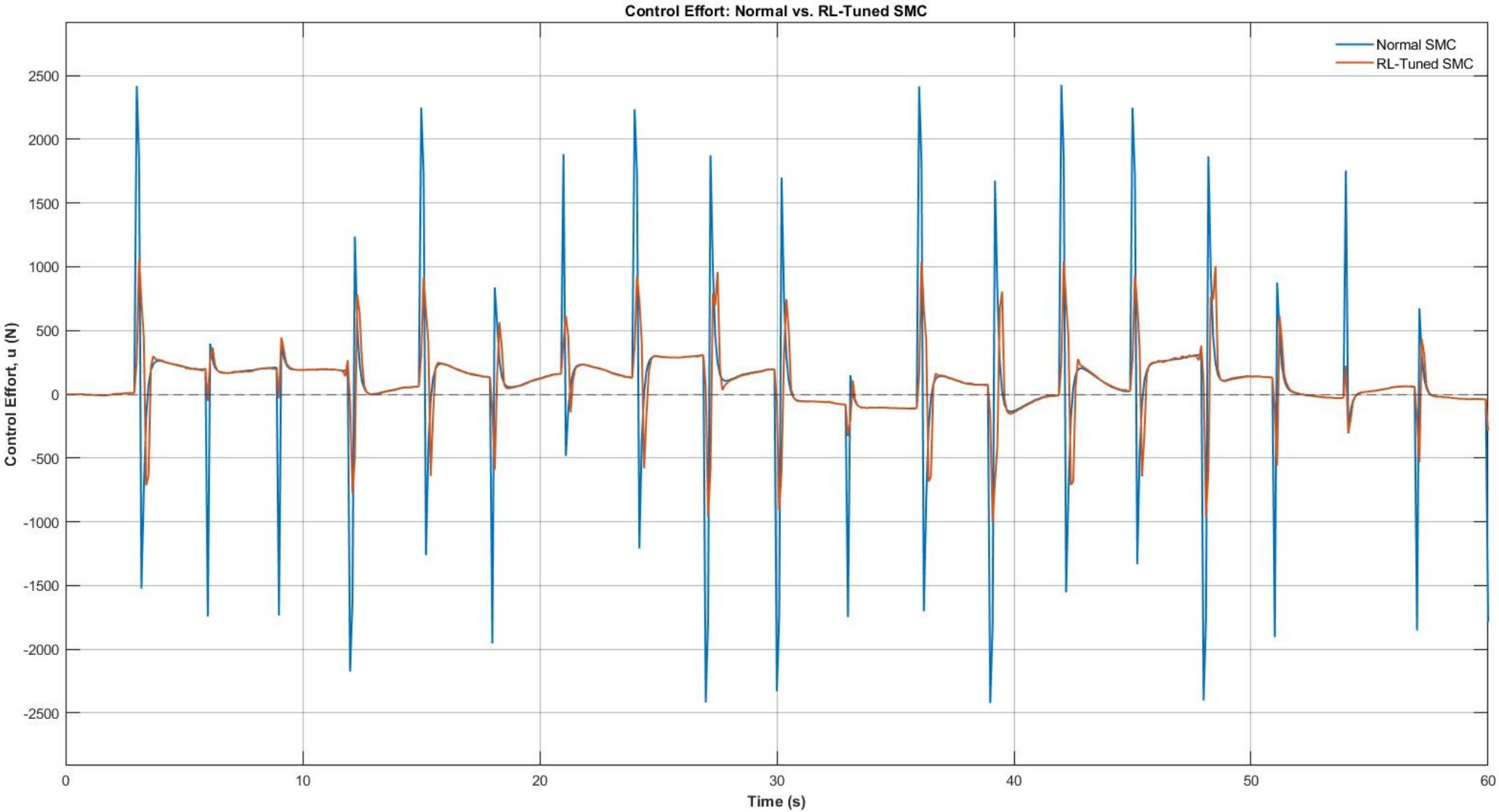


Figure 5.14: Meta-RL SMC vs Normal SMC's energy usage

d. Proof of Stability

Lyapunov stability:

A Lyapunov stability proof is essentially a mathematical argument that a system won't get out of control. The idea comes from Aleksandr Lyapunov, who showed that if one can find a special energy-like function — called a **Lyapunov function** V(x) — that satisfies two conditions, one can prove the system is stable without solving the differential equations directly:

1. **V(x) > 0** — the function is always positive (like energy, it's never negative)
2. **$\dot{V}(x) \leq 0$** — the function is always decreasing over time (the "energy" is always draining away, never growing)

Since our RL agent outputs parameter adjustments with bounded ranges (already defined ranges η: 50→150, ϕ: 0.030→0.170), one may argue:

The RL agent's output is constrained to $[\eta_{min};\ \eta_{max}]$. For all $\eta \geq \eta_{min}$ the standard SMC Lyapunov condition $\dot{V} = s \times \dot{s} \leq -\eta|s| < 0$ holds. Therefore, stability is preserved regardless of which specific value the agent selects. However, this is only true when manually setting the parameters at the beginning of a simulation run, not for something that switches these parameters on the fly.

If our parameters are perfectly static, $\dot{V} =\leq -\eta|s|$ holds true, and the system is stable. But an RL agent makes those parameters dynamic, essentially turning those parameters into time-varying functions.

When the AI tunes everything simultaneously, it is essentially moving the goalpost (altering the sliding surface s via C_2, C_4) while simultaneously changing how hard the system runs toward it (altering the reaching law via η) and how soft the landing pad is (altering the boundary layer ϕ).

This in turn makes the math intractable, that is to say this might require multi-page mathematical tangles to crack a certain problem which could be solved by arguing the system is inherently stable empirically, via the Monte Carlo results should 300 benchmarking episodes at 60s each be considered "satisfactory" — which it might not be for stricter grading rubrics or for more complex systems.

Daniel Liberzon's 2003 book, "Switching in Systems and Control":

Imagine having two completely stable controllers. Liberzon points out a reality: You can take two perfectly stable systems, switch between them at the wrong times, and create a completely unstable, explosive system.

Think of a person on a swing. If you push them at the exact right moment (Stable System A), they swing safely. On the other hand, if you pull them at the exact right moment (Stable System B), they also swing safely. But if you rapidly switch between pushing and pulling at the wrong frequency, the energy builds up erratically, and the person gets thrown into the air.

In this assignment, every time the Meta-RL agent outputs a new set of [C_2, C_4, η, φ] it is instantly switching the physical system to a new mathematical state. Even if every individual state is stable, the

sheer act of "snapping" the parameters creates a sudden spike in energy. If the AI switches parameters too fast, those energy spikes accumulate, and the sliding surface destabilizes.

Now let us focus on the "too fast" framing, and the fact that SMC is theoretically stable for controllers whose parameters are not changed (or at least not having all its significant parameters changed) as the controller is online. For this reason solely we may argue that: from the controller's 50Hz control clock and the simulation's 100Hz clock, during the 60 seconds of simulation, there are 3000 separate controller schemes which individually satisfy the standard SMC Lyapunov condition of $\dot{V} = s \times \dot{s} \leq -\eta|s| < 0$.

Liberzon's Solution: Dwell-Time (τ_d)

To guarantee stability in a switched system, Liberzon proved that one must enforce a "speed limit" on how fast the system is allowed to change states. This is called Dwell-Time.

The system must "dwell" in its current state long enough to naturally dissipate the energy spike caused by the previous switch. With the dwell time as specified by:

$$\tau_d = \frac{\ln(\mu)}{\lambda}$$

Liberzon's proof relies on two competing mathematical forces:

1. **The Energy Decay (λ):** While the parameters are locked, SMC naturally drains energy from the system (pushing the error to zero). We call this decay rate $\boldsymbol{\lambda}$**.**
2. **The Switching Jump (μ):** When the AI changes C_2, C_4, the sliding surface *s* shifts instantly. This causes the Lyapunov energy V(x) to suddenly jump. We assume the energy jumps by a maximum multiplier of μ (where $\mu \geq 1$). μ is simply the ratio of the energy immediately *after* the switch, compared to the energy immediately *before* the switch. The formula is $\mu = \frac{V^+}{V}$.
   With $V(x) = \frac{1}{2}s^2$, we have $V^+(x) = \frac{1}{2}(s + \Delta s)^2$; so $\mu = \left(\frac{s+\Delta s}{s}\right)^2 = \left(1 + \frac{\Delta s}{s}\right)^2$

To apply Liberzon's dwell-time theorem, we must first prove that as long as the AI's parameters are locked (between the 0.02s control updates), the system exhibits exponential energy decay.

We begin with the standard SMC Lyapunov candidate function:

$$V(x) = \frac{1}{2}s^2$$

Taking the time derivative yields $\dot{V} = s\dot{s}$. The sliding surface dynamics utilizing a smooth hyperbolic tangent boundary layer are defined as:

$$\dot{s} = -\eta \tanh\left(\frac{s}{\phi}\right)$$

Substituting this into the Lyapunov derivative gives:

$$\dot{V} = s \times -\eta \tanh\left(\frac{s}{\phi}\right)$$

To satisfy the exponential decay condition $\dot{V} \leq -\lambda V$, we must find a constant $\lambda > 0$ such that:

$$-\eta s \tanh\left(\frac{s}{\phi}\right) \leq -\lambda\left(\frac{1}{2}s^2\right)$$

$$\lambda \leq \frac{2\eta}{s}\tanh\left(\frac{s}{\phi}\right) = \frac{2\eta}{\phi}\left(\frac{\phi}{s}\right)\tanh\left(\frac{s}{\phi}\right) = \frac{2\eta}{\phi}\left[\frac{\tanh\,(s/\phi)}{(s/\phi)}\right]$$

The term $\left[\frac{\tanh\,(s/\phi)}{(s/\phi)}\right]$ maxes out at exactly 1 as $\left(\frac{s}{\phi}\right) \to 0$; and bottoms out at 0 as $\left(\frac{s}{\phi}\right) \to \pm\infty$

Let $z = (s/\phi)$. Because we are analyzing the worst-case decay inside the designated boundary layer (where $|z| \leq 1$), the minimum value of this function occurs at the boundary edge ($|z| = 1$):

$$\frac{\tanh\,(1)}{1} \approx 0.7616$$

Therefore, the guaranteed exponential decay rate for the smooth tanh reaching law is:

$$\lambda = 0.7616\left(\frac{2\eta}{\phi}\right)$$

Given the RL agent's strict parameter boundaries of $\eta_{min} = 50;\ \phi_{max} = 0.170$, we can calculate the absolute worst-case decay rate:

$$\lambda_{min} = 0.7616\left(\frac{2(50)}{0.170}\right) \approx 448.0$$

Because the AI's is clocked at 0.02s:

$$0.02 \geq \frac{\ln\,(\mu_{max})}{448.0}$$

<u>Calculating μ_{max}</u>

Step 1: The Maximum Parameter Shift ΔC

In `calc_reward`, there exists a chatter penalty that triggers if the total AI action shift $\Delta u_{sum} > 0.2$. Let's assume the worst-case legal move where the AI shifts its output by exactly 0.2 in a single 0.02s frame.

$C_2 = 20 + 5.0(u_1)$. A 0.2 shift in u_1 means $\Delta C_2 = 5.0 \times 0.2 = 1.0$

$C_4 = 2 + 1.0(u_1)$. A 0.2 shift in u_2 means $\Delta C_1 = 1.0 \times 0.2 = 0.2$

Step 2: The Maximum Physical State x_{max}

The sliding surface is multiplied by the physical states (positions and velocities). We must assume the switch happens at the absolute limits of the physical hardware:

Maximum distance: $x_{2\ max}$ = 0.80 m.

Maximum velocity: $v_{2\ max}$ = 2.5 m/s.

Step 3: The Maximum Surface Shift Δs

Assuming a standard sliding surface structure ($s = C^T x$), the maximum instantaneous physical shift of the goalpost is:

$$\Delta s_{max} = |\Delta C_2 \times x_{2\ max}| + |\Delta C_4 \times v_{2\ max}|$$

$$\Delta s_{max} = |1 \times 0.8| + |0.2 \times 2.5| = 1.3$$

Step 4: Calculating μ_{max}

Now we calculate the Switching Shock (μ) at the most vulnerable point: the edge of the boundary layer $\phi_{min} = 0.030$:

$$\mu_{max} = \left(1 + \frac{\Delta s_{max}}{\phi_{min}}\right)^2 = \left(1 + \frac{1.30}{0.030}\right)^2 \approx 1965.4$$

The Final Dwell-Time Calculation:

$$0.02 \geq \frac{\ln(\mu_{max})}{448.0} = \frac{\ln(1965.4)}{448.0}$$

$$0.02 \geq \frac{7.583}{448.0}$$

$$0.02 \geq 0.0169\ (seconds)$$

Because the RL agent's macro-clock ticks at exactly 0.02 seconds (50Hz), the agent is physically incapable of switching parameters faster than the required dwell-time. The system is then mathematically proven to be "**Globally Asymptotically Stable**".

Meaning: no matter how extreme a system's initial starting conditions are, it is guaranteed to eventually settle down and come to a complete rest exactly at its desired equilibrium point.

VII. The Pros and Cons of RL Agent Training

Cons: Why Many Industries Reject It

1. Hardware demands: Training a TD3 agent already requires a dedicated GPU, substantial RAM, and significant storage for the replay buffer alone (around 250GB for this 5000-step Agent alone!). For industries relying on lean and deterministic embedded systems, like automotive ECUs, aerospace flight computers, industrial PLCs, the inference hardware alone may be disqualifying, let alone the training infrastructure. This is simply forbidding for the majority of university students.

2. Training time: Every time the designer alters the system with new payload, different spring constants, revised actuator limits, the agent must be retrained. A classical SMC could have its sliding surfaces reiterated in less than a day. The RL designer must re-run a 12-hour GPU job and re-validate with the Monte Carlo test. In fast-iteration development cycles, this is often unacceptable.

3. Retuning is retraining: Classical controllers have direct, tunable parameters: increase this number to get a faster response, decrease this number to achieve a lesser average energy consumption. The RL agent has none. In this case, tracking performance is 33% short of the requirement, the only intervention is to reshape the reward function and retrain or at least continue from Agent 5000, which is a process that will most likely converge to an entirely different policy.

4. The validation problem: For example, when the agent commands $u = 2300$N at $t = 45.8$s instead of $u = 1200$N, there is no "decision making" chain to follow. Therefore, **Certification is nearly impossible.** Aviation (DO-178C), medical (IEC 62304), and automotive (ISO 26262) safety standards require engineers to demonstrate and explain correct behavior under all conditions. "The network converged upon this result" is not an acceptable justification.

Then Why Bother At All?

All four cons above share an assumption: that the system is well-understood, stable over time, and operates within pre-specified conditions. And that the controller is in charge of a system whose malfunctions could lead to catastrophic loss of assets, or lives.

When the environment is too complex to model accurately, the RL agent's learned generalization becomes valuable here first. Classical SMC and MPC degrade when their underlying model is significantly wrong. The RL agent was never given a model, instead it learned the system's actual behavior across millions of sampled conditions, therefore making model mismatch a condition it has likely already encountered rather than a failure mode.

When the performance trade-off is the problem: No fixed algorithm sits optimally across all axes simultaneously. The RL agent navigates the Pareto surface in real time, because it has experienced the consequences of every trade-off across millions of episodes and encoded that experience into its weights.

When hardware survival matters more than absolute tracking: The benchmark result shows: Meta-RL SMC achieved half the peak actuator stress of fixed SMC for essentially the same average energy expenditure. In systems where actuator replacement/system downtime is expensive, dangerous, or

impossible (e.g.: offshore machinery, deep-space mechanisms, implanted medical devices), a controller that conserves mechanical life at the cost of some steady-state precision becomes a requirement.

➔ **The bottom line:** The RL Meta-SMC is not a universally better controller. It is a different tool for a different class of problems, problems where the model is uncertain, the environment is every-changing, and performance trade-offs are generally unacceptable. For those problems, the training cost, hardware overhead, and ambiguity in decision penalties are “elevated” from being merely demands, to being the price of admission to a solution that should far exceed classical algorithms.